

Java Exercise Book

Piotr Jastrzębski

2026-11-03

Table of contents

Java Exercise Book 4

I Introduction to Java 5

1 Input/Output Operations 6

2 Conditional Statements 7

3 Loops 9

4 Functions/Methods 12

5 Generating Pseudorandom Numbers 14

6 Arrays 15

7 ArrayList (Array Lists) 18

8 Strings 20

II Object-Oriented Programming 22

9 The Concept of a Class/Object 23

10 Access Modifiers 26

11 Constructor 27

12 Static Fields and Methods 31

13 Records 33

14 The `toString`, `equals` and `hashCode` Methods 35

15 Inheritance 40

16 Packages 44

17 Complex Fields in a Class 46

18 Abstract Classes 48

19 Final Fields, Methods and Classes 50

20 The `Comparable` Interface 52

21 The `Comparator` Interface 56

22 Copying Objects 59

23 Interfaces 62

24 Exceptions 66

25 Generic Programming 67

26 Collections 71
- 26.1 The `Collection` Interface . . . 71
- 26.2 The `Iterable` Interface . . . 71
- 26.3 The `ArrayList` Array List . . . 71
- 26.4 The `LinkedList` Linked List . . . 72
- 26.5 The Hash-Table-Based Set `HashSet` . . . 72
- 26.6 `TreeSet` . . . 72
- 26.7 `Queue` . . . 73
- 26.8 `Deque` . . . 73
- 26.9 The `PriorityQueue` Priority Queue . . . 74
- 26.10`Map` . . . 74
- 26.11`HashMap` . . . 74
- 26.12`TreeMap` . . . 75
- 26.13`Vector` . . . 75

27 Delegation 76

III Miscellaneous Exercises 77

IV Design Patterns 78

References and Other Exercise Collections 79

Java Exercise Book

This is a collection of Java programming exercises. It grew out of exercises created while teaching classes at the Faculty of Mathematics and Computer Science of the University of Warmia and Mazury in Olsztyn.

Some of the exercises were generated with ChatGPT, Claude, Gemini or DeepSeek.

Part I

Introduction to Java

1 Input/Output Operations

1. Write a simple text calculator application that takes two numbers from the user as input and performs basic mathematical operations (addition, subtraction, multiplication, division). Display the results on the screen.
2. Write a program that reads two strings from standard input and then prints them on consecutive lines to standard output.
3. Write a program that reads four rational numbers from standard input and then prints their sum to standard output.
4. Create a program that calculates the area of a square. Get the data from the user and display the result on standard output.
5. Write a program in which you read the values of the variables from the user, then compute and display the results of the operations:

- $a + b - x^2$
- $\frac{a-b}{c-3}$
- $3(4 + 5a)(b - c^3)$

6. Write a program in which the following operations are performed using so-called compound assignment operators (`+=` , `-=` and other similar operators):

- $a = a + 4$
- $b = b - a$
- $c = c(2 - 4a)$
- $d = \frac{d}{4-a^2}$

2 Conditional Statements

1. Write a program that checks whether a given integer is even. If it is, the program should print "The number is even", otherwise "The number is odd".
2. Write a program that takes three integers as arguments and returns the largest of them. Use conditional statements to compare the numbers.
3. Write a program that, based on a day-of-the-week number (from 1 to 7) given as an argument, prints the name of that day of the week. For example, if the user enters 1, the program should print "Monday". If the given number is not in the range from 1 to 7, the program should display the message "Invalid day-of-the-week number".
4. Write a program that solves a quadratic equation of the form $ax^2 + bx + c = 0$. The program should take the values a, b, and c as arguments, compute the discriminant $(b^2 - 4ac)$, and then return the solutions of the equation depending on the value of the discriminant.
5. Write a program that takes the user's age as an argument. If the age is less than 18, the program should display "You are a minor". If the age is greater than or equal to 18 but less than 65, the program should display "You are an adult". If the age is equal to or greater than 65, the program should display "You are a senior".
6. Write a program that checks whether a given year is a leap year. A year is a leap year if it is divisible by 4 but not divisible by 100, unless it is divisible by 400.
7. Write a program that takes three integers as arguments and sorts them in ascending order using conditional statements, and then displays the sorted numbers.
8. Write a program that calculates income tax based on the given income and tax rules. Assume that the tax is 18% for income up to 85,528 PLN, and for income above that amount the tax is 14,839.02 PLN plus 32% of the excess over 85,528 PLN. The user should enter their income, and the program should calculate and display the tax amount.
9. Write a program that checks whether numbers entered from the keyboard can form a valid calendar date.

Sample input:

```
Enter day: 29
Enter month: 2
Enter year: 2017
```

Sample output:

```
Invalid date
```

10. Write a program that checks whether numbers entered from the keyboard can form a valid time in 24-hour format.

Sample input:

```
Enter hour: 22
Enter minutes: 12
Enter seconds: 33
```

Sample output:

```
Valid time!
```

11. Write a program in which the user enters a three-digit integer. Then check whether the number is a palindrome. Display an appropriate message on the console.
12. Write a program that reads three integers (in theory they may have different signs) and checks whether they can form a right triangle - finally it prints “YES” or “NO”.

3 Loops

1. Write a program that uses a `for` loop to display the numbers from 1 to 100.
2. Write a program that uses a `while` loop to calculate the sum of the numbers from 1 to 50.
3. Write a Java program that uses a `for` loop to generate the first 10 numbers of the Fibonacci sequence.
4. Create a program that uses nested `for` loops to display the multiplication table for the numbers from 1 to 10.
5. Write a program that uses a `do-while` loop to display the first 20 even and odd numbers.
6. Write a program that checks whether a given number is prime. The number should be entered by the user.
7. Write a program that reads a sequence of 10 integers from the user and then displays the largest and the smallest of them.
8. Write a program that calculates the sum of the digits of any entered number. The program should accept the number as input from the user.
9. Write a program that generates and displays the first 10 terms of a geometric series with a given first term and common ratio. The parameters of the series should be entered by the user.
10. Create a program that takes an integer from the user and returns the number with its digits in reverse order. For example, for the number 12345, the result should be 54321. You may restrict the program to positive numbers only.
11. Write a program that calculates the sum of the squares of the numbers from 1 to n, where n is a number entered by the user.
12. Write a program that finds and displays all perfect numbers less than 10,000. A perfect number is a number whose sum of divisors (excluding the number itself) is equal to its value. For example, 6 is a perfect number because $1 + 2 + 3 = 6$.
13. Write a program that finds all Armstrong numbers less than 10,000. An Armstrong number is a number for which the sum of its digits, each raised to the power equal to the number of digits in the number, is equal to the number itself. For example, 153 is an Armstrong number because $1^3 + 5^3 + 3^3 = 153$.

14. Write a program that calculates the least common multiple (LCM) of two given numbers. The user should enter two numbers as input.

15. Write a program that reads a positive integer from the keyboard. Then draw the corresponding triangle, e.g. for 5:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

15. Write a program that reads a natural number n ($n \geqslant 0$) from the user. Then determine and display the n-th term of the sequence:

a) $(4, -8, 16, -32, 64, ...)$
b) $(2, 6, 18, 54, 162, ...)$
c) $(8, 3, -2, -7, -12, ...)$

16. Write a program that reads consecutive integers (of various signs) from the keyboard and stops when their sum exceeds 100.

17. Write a program that reads a positive integer n from the user, and then n integers $a_1, ..., a_n$. The program should display how many of these numbers satisfy the condition $3^k < a_k < k!$ for $1 \leq k \leq n$.

18. Write a program that displays the first sixteen powers of two on the screen. Use loops for this purpose.

19. Write a program that reads 5 integers from the user. Reading should be interrupted when the user enters a negative number. If reading is not interrupted, display the sum of the entered numbers.

20. Write a program that reads two integers n and m from standard input (we assume that $n < m$) and prints the value of $n \cdot ... \cdot m$ to standard output.

21. Write a program that reads two integers n and m from standard input (we assume that $n < m$) and prints the value of $n + ... + m$ to standard output.

22. Write a program that reads three positive integers a, b, c from the user. On standard output, display the positive integers greater than b, less than or equal to a, and divisible by c.

23. Write a program that reads a natural number a from standard input and then prints to standard output how many of the digits of a are equal to 7.

24. Write a program that reads integers from input until a negative number is encountered, and then displays the largest and the smallest of the numbers read (excluding the last, negative number).

25. Write a program that asks the user to enter a positive integer `n`, and then calculates and displays the largest natural number `k` such that $k!$ (factorial) does not exceed `n`. The task must be solved without arrays, strings, or built-in mathematical functions. Do not write your own functions either.

Example: for `n=25` the program should display 4 (because $4! = 24 \leqslant 25$, but $5! = 120 > 25$), and for `n=120` it should display 5.

4 Functions/Methods

1. Write a static method whose argument is a positive integer n. The method should return, as an integer, the sum of the series

$$1 - 2 + 3 - 4 + ... \pm n.$$

Create a test case for this method.

2. Write a static method whose argument is a positive integer n ($n > 2$). The method should return the largest prime number less than n. Create a test case for this method.

3. Write a static method whose argument is a positive integer n. The method returns the appropriate boolean value indicating whether n is a perfect number. A perfect number is a number whose sum of divisors is equal to the number itself (examples of perfect numbers are $1 = 1$, $6 = 1 + 2 + 3$). Create a test case for the method.

4. Write a static method whose parameters are two positive integers a and b. The method should return the least common multiple (LCM) of a and b. Create a test case for the method.

Example: $LCM(5, 10) = 10$, $LCM(4, 5) = 20$.

5. Write a static method whose argument is a positive integer n. The method returns `true` if the given number n is odd, positive, consists of 4 digits and is divisible by 5, and returns `false` otherwise. Create a test case for the method.

6. Write a static method that receives a positive integer n as an argument and returns the number 7^{-n}. Do not use any ready-made library or built-in functions inside this function other than input/output statements. Create a test case.

Hint: $7^{-n} = \frac{1}{7^n}$.

7. Write a static method whose arguments are four positive integers a, b, c, d. The method returns how many integers in the interval (a, b) are divisible by c and not divisible by d. If there are no such numbers, return zero. Create a test case for this method.

8. Write a static method whose argument is a positive integer n. The method returns the sum of the integers from n to $2n$ (inclusive). Create a test case for the method.

9. Write a static method whose arguments are non-negative integers n and k. The method returns the value of the expression:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Create a test case for the method.

10. Write a static method whose argument is a positive integer n. The method returns 1 if n is a number consisting only of ones in decimal notation and returns 0 otherwise. Create a test case for the method.

11. Write a static recursive method `quadSeq` that, for a non-negative integer n received as an argument, returns the value of the term with index n of the sequence defined as follows:

$$v_0 = 1$$

$$v_n = \begin{cases} v_{n-1} + n, & \text{if } n \text{ is divisible by 4} \\ v_{n-1} \cdot 2, & \text{if } n \text{ leaves remainder 1 when divided by 4} \\ v_{n-1}^2 - 1, & \text{if } n \text{ leaves remainder 2 when divided by 4} \\ \lceil \frac{v_{n-1}}{3} \rceil, & \text{if } n \text{ leaves remainder 3 when divided by 4} \end{cases}$$

where $\lceil x \rceil$ denotes the ceiling function (the smallest integer not less than x). Do not use built-in mathematical functions in the solution. Create a test case for the function.

n	0	1	2	3	4	5	6
v_n	1	2	3	1	5	10	99

5 Generating Pseudorandom Numbers

1. Write a method `generateRandomInt` that generates and returns a random integer. Create a test case.
2. Write a method `generateRandomDouble` that generates and returns a random floating-point number in the range from 0.0 to 1.0. Create a test case.
3. Write a method `generateRandomIntInRange` that takes two integers as arguments and returns a random integer from that range (inclusive of the bounds). For example, for the arguments 5 and 10, the method should return a number in the range from 5 to 10. Create a test case.
4. Write a method `generateRandomGaussian` that generates and returns a random floating-point number following the normal distribution. Create a test case.
5. Write a method `generateRandomBoolean` that generates and returns a random boolean value (true or false). Create a test case.

6 Arrays

1. Write a program that creates a one-dimensional array of 10 integers and then displays them on the console in the reverse order of input.
2. Create a program that creates a one-dimensional array of 20 random numbers in the range from 1 to 100, and then calculates and displays their average value.
3. Write a program that creates a one-dimensional array of 15 integers, and then calculates and displays the largest and the smallest value in the array.
4. Create a program that creates a one-dimensional array of 30 integers. Then ask the user to enter any number. The program should display whether the given number is in the array, and also how many times it appears in it.
5. Write a program that creates a one-dimensional array of 10 integers. The program should calculate and display the sum of those numbers that are even.
6. Write a program that creates a one-dimensional array of 10 floating-point numbers, and then calculates and displays their sum.
7. Create a program that creates a one-dimensional array of 20 floating-point numbers, and then finds and displays the mean and the median of these numbers.
8. Write a program that creates a one-dimensional array of 10 floating-point numbers. The program should calculate and display the sum of those numbers that are greater than 0.5.
9. Write a program that creates a one-dimensional array of 20 integers, and then calculates and displays the number of even and odd numbers in the array.
10. Create a program that creates a one-dimensional array of 30 integers. Then the program should calculate and display how many of the numbers are squares of another integer.
11. Write a program that creates a one-dimensional array of 50 integers. The program should calculate and display how many of the numbers are prime numbers.
12. Write a program that creates a one-dimensional array of 15 integers, and then calculates and displays the sum of the numbers that are divisible by 3.
13. Write a static method `minimumValue` that takes an array of integers as an argument and returns the smallest number in the array. Assume that the array will always have at least one element. If the array contains only one element, return that element. Create a test case.

14. Write a static method `average` that takes an array of floating-point numbers as an argument and returns the arithmetic mean of all the numbers in the array. If the array is empty, return 0. Create a test case.

15. Write a static method `reverseArray` that takes an array of integers as an argument and returns a new array, but with the order of the elements reversed. For example, for the array [1, 2, 3, 4, 5], the function should return [5, 4, 3, 2, 1]. Create a test case.

16. Write a static method `reverseArray` that takes an array of integers as an argument. The method reverses the order of the elements in the array and should be a procedure (return nothing). For example, for the array [1, 2, 3, 4, 5], the function should change the array to [5, 4, 3, 2, 1]. Create a test case.

17. Write a static method `countZeros` that takes an array of integers as an argument and returns the number of occurrences of zero in the array. For example, for the array [0, 1, 2, 0, 3, 0, 4], the function should return 3. Create a test case.

18. Write a static method `oddElementsSum` that takes an array of integers as an argument and returns the sum of all odd numbers in the array. If there are no odd numbers in the array, the function should return 0. Create a test case.

19. Write a static method `copyArray` that takes an array of integers as an argument and returns a new array that is a copy of the original. The function should copy the array manually, element by element, without using the system copy methods. Create a test case.

20. Write a static method `copyArrayReverse` that takes an array of integers as an argument and returns a new array that is a reversed copy of the original. For example, for the array [1, 2, 3, 4, 5], the function should return [5, 4, 3, 2, 1]. Create a test case.

21. Write a static method `copyArrayEven` that takes an array of integers as an argument and returns a new array containing only the even numbers from the original array. For example, for the array [1, 2, 3, 4, 5], the function should return [2, 4]. If there are no even numbers in the array, the function should return an empty array. Create a test case.

22. Write a static method `copyArrayWithIndex` that takes an array of integers and a start and end index as arguments. The function should return a new array containing the elements of the original array from the start index to the end index (inclusive). For example, for the array [1, 2, 3, 4, 5] and the indices 1 and 3, the function should return [2, 3, 4]. Create a test case.

23. Write a static method `mergeArrays` that takes two arrays of integers as arguments. The function should return a new array that is the concatenation of both original arrays. For example, for the arrays [1, 2, 3] and [4, 5, 6], the function should return [1, 2, 3, 4, 5, 6]. Create a test case.

24. Write a method `sortArray` that takes an array of integers as an argument and returns a new array that is a sorted version of the original array. Use the `Arrays.sort()` method from the `java.util.Arrays` library to sort the array. Create a test case.

25. Write a method `checkEquality` that takes two arrays of integers as arguments and returns `true` if the arrays are equal and `false` otherwise. Use the `Arrays.equals()` method from the `java.util.Arrays` library to compare the arrays. Create a test case.

26. Write a method `fillArray` that takes an array of integers and an integer as arguments. The method should fill the array with the given number, using the `Arrays.fill()` method from the `java.util.Arrays` library. Create a test case.

27. Write a method `printArray` that takes an array of integers as an argument and prints its contents to the console. Use the `Arrays.toString()` method from the `java.util.Arrays` library, which returns a textual representation of the array. Create a test case.

28. Write a method `copyArray` that takes an array of rational numbers as an argument. The method should return a new array that is a copy of the passed array. Use the `Arrays.copyOf()` method from the `java.util.Arrays` library to copy the array. Create a test case.

29. Write a method `sortArray` that takes an array of rational numbers as an argument. The method should sort the array in ascending order, using the `Arrays.sort()` method from the `java.util.Arrays` library. Create a test case.

7 ArrayList (Array Lists)

1. Write a program that creates an array list of 10 integers and then displays them on the console in the reverse order of input.
2. Write a program that creates an array list of 10 floating-point numbers, and then calculates and displays their sum.
3. Write a static method `minimumValue` that takes an array list of integers as an argument and returns the smallest number in the array list. Assume that the array list will always have at least one element. Create a test case.
4. Write a static method `minimumValue` that takes an array list of rational numbers as an argument and returns the smallest number in the array list. Assume that the array list will always have at least one element. Create a test case.
5. Write a static method `average` that takes an array list of floating-point numbers as an argument and returns the arithmetic mean of all the numbers. If the array list is empty, return 0. Create a test case.
6. Write a static method `reverseArray` that takes an array list of integers as an argument and returns a new array list, but with the order of the elements reversed. For example, for [1, 2, 3, 4, 5], the function should return [5, 4, 3, 2, 1]. Create a test case.
7. Write a static method `reverseArray` that takes an array list of integers as an argument. The method reverses the order of the elements in the array list and should be a procedure (return nothing). For example, for [1, 2, 3, 4, 5], the function should change the array list to [5, 4, 3, 2, 1]. Create a test case.
8. Write a static method `countZeros` that takes an array list of integers as an argument and returns the number of occurrences of zero in the array list. For example, for [0, 1, 2, 0, 3, 0, 4], the function should return 3. Create a test case.
9. Write a static method `oddElementsSum` that takes an array list of integers as an argument and returns the sum of all odd numbers in the array list. If there are no odd numbers in the array list, the function should return 0. Create a test case.
10. Write a static method `copyArray` that takes an array list of integers as an argument and returns a new array list that is a copy of the original. The function should copy the array list manually, element by element, without using the system copy methods. Create a test case.

11. Write a static method `copyArrayReverse` that takes an array list of integers as an argument and returns a new array list that is a reversed copy of the original. For example, for the array list [1, 2, 3, 4, 5], the function should return [5, 4, 3, 2, 1]. Create a test case.

12. Write a static method `copyArrayEven` that takes an array list of integers as an argument and returns a new array list containing only the even numbers from the original array list. For example, for the array list [1, 2, 3, 4, 5], the function should return [2, 4]. If there are no even numbers in the array list, the function should return an empty array list. Create a test case.

13. Write a static method `copyArrayWithIndex` that takes an array list of integers and a start and end index as arguments. The function should return a new array list containing the elements of the original array list from the start index to the end index (inclusive). For example, for the array list [1, 2, 3, 4, 5] and the indices 1 and 3, the function should return [2, 3, 4]. Create a test case.

14. Write a static method `mergeArrays` that takes two arrays of integers as arguments. The function should return a new array list that is the concatenation of both original arrays. For example, for the array lists [1, 2, 3] and [4, 5, 6], the function should return [1, 2, 3, 4, 5, 6]. Create a test case.

15. Write a method `sortArray` that takes an array list of integers as an argument and returns a new array list that is a sorted version of the original array list. Create a test case.

8 Strings

1. Write a static method that takes a string as an argument and returns the string in reverse order. Create a test case.
2. Write a static method that checks whether a given string is a palindrome. A palindrome is a word, phrase, number or other sequence of characters that reads the same forwards and backwards. Create a test case.
3. Write a static method that removes all spaces from a given string. Create a test case.
4. Write a static method that takes a string as an argument and returns the string with its first and last letters swapped. Create a test case.
5. Write a program that analyzes a given string for the frequency of occurrence of each character. The program should display each character and the number of its occurrences in the given string. Read the data from standard input.
6. Write a program that takes a string as input and prints all the characters located at even indices of the string, using the `charAt` method.
7. Using the `charAt` method, write a static method that checks whether a given string starts and ends with the same character. Create a test case.
8. Write a program that takes three strings: a main string, a prefix and a suffix. Using the `startsWith` and `endsWith` methods, check whether the main string starts with the given prefix and ends with the given suffix. Print an appropriate message for each of these cases.
9. Write a program that takes a single character and an integer `n` as input. Using the `StringBuilder` class, build and print a pyramid of height `n`, where each level of the pyramid consists of the given character. For example, for the character `*` and `n=3`, the expected output is:

```
  *
 ***
*****
```

10. A string is given. Using `StringBuilder`, write a program that removes all repeated characters, leaving only the first occurrence of each character. For example, for the string "banana", the expected result is "ban".

11. Write a program that takes a sentence as input. Using `StringBuilder`, reverse each word in the sentence, but keep the order of the words. For example, for the sentence "Java is fun", the expected result is "avaJ si nuf".

12. Write a static method `reverseString` that takes a `StringBuilder` object as an argument and returns a new `StringBuilder` that is the reversed original string. Create a test case.

13. Write a static method `removeDuplicates` that takes a `StringBuilder` and removes all repeated characters in the string, leaving only the first occurrence of each character. Create a test case.

14. Write a static method `mostFrequentChar` that takes a `StringBuilder` as an argument and returns the character that occurs most frequently in the string. In case of ties, return the first of the tied characters. Create a test case.

15. Write a static method `insertSubstring` that takes two arguments: `StringBuilder sb` and `String toInsert`. The method should insert `toInsert` into the middle of the original `StringBuilder`. Create a test case.

16. Write a static method `splitByEvenOdd` that takes a `StringBuilder` as an argument. The method should return an array of two elements of type `StringBuilder`. The first element of the array should contain the characters from the odd indices of the original string, and the second the characters from the even indices. Create a test case.

17. Write a static method `capitalizeEverySecond` that takes a `StringBuffer` object as an argument. The method should change every second letter to uppercase. Create a test case.

18. Write a static method `replaceSubstring` that takes the arguments `StringBuffer sb`, `String oldSub` and `String newSub`. The method should replace all occurrences of the substring `oldSub` with `newSub`. Create a test case.

19. Write a static method `countOccurrences` that takes two arguments: `StringBuffer sb` and `char c`. The method should return the number of occurrences of the character `c` in the string. Create a test case.

20. Write a static method `trimToSize` that takes a `StringBuffer sb` and an integer `n`. The method should reduce the length of the string to `n` characters (if the original string is longer). If the string is shorter than or equal to `n`, it should remain unchanged. Create a test case.

21. Write a static method `isPalindrome` that takes a `StringBuffer` as an argument. The method should check whether the string is a palindrome (reads the same forwards and backwards) and return the appropriate boolean value. Create a test case.

Part II

Object-Oriented Programming

9 The Concept of a Class/Object

1. Create a class `Dog` with the fields `name`, `breed` and `age`. Write a method `bark()` that prints `"Wow Wow"` to the console. Create a test case to call the method at least once.
2. Create a class `Car` with the fields `brand`, `model` and `speed`. Write the methods `accelerate(int value)` and `decelerate(int value)`, which respectively increase and decrease the speed by the given value. Create a test case to call each method at least once.
3. Create a class `BankAccount` with the field `balance`. Write the methods `deposit(double amount)` and `withdraw(double amount)`, which respectively increase and decrease the balance by the given amount. Create a test case to call each method at least once.
4. Create a class `Point` with two fields, `x` and `y`, representing coordinates on a plane. Write a method `distance(Point otherPoint)` that calculates the distance between the current point and another point. Create a test case to call the method at least once.
5. Create a class `Time` with the fields `hours` and `minutes`. Write a method `addTime(Time otherTime)` that adds the time given as an argument to the current time and returns a new object of the Time class. Make sure that the minutes and hours do not exceed 59 and 23, respectively. Create a test case to call the method at least once.
6. Complete the two subtasks in order:

   A. Create a class `Car` containing a public field `brand`. In the `TestCar` class, create an object of the `Car` class, assign it the value `null` and try to access the `brand` field. What is the result?

   B. Modify the code from subtask A so that it checks whether the `Car` object is `null` before accessing the `brand` field. What is the result?
7. Create a class `Person` containing the field `name`. In the `TestPerson` class, create two variables of the `Person` class - `person1` and `person2` - both referring to the same object. Change the value of the `name` field through `person1` and print the value of the `name` field through `person2`. Analyze the result.
8. Create an array of objects of the `Car` class (the class contains a public field `brand`). Then try to access the `brand` field of one of the objects in the array without first initializing the array with `Car` objects. Analyze the result.

9. Create an array list (ArrayList) of objects of the `Person` class (the class contains a public field `name`). Then try to access the `name` field of one of the objects in the list without first adding any `Person` objects to the list. Analyze the result.

10. Create a class `Dog` with a method `bark` that prints the message "Woof! Woof!". In the `TestDog` class, create a `Dog` object, assign it the value `null` and try to call the `bark` method. Analyze the result.

11. Create a class `Cat` with the field `name`. Add to the `Cat` class a method `createCat` that returns a new object of the `Cat` class. The method should set the `name` field to the given value, but only if the value is not `null`. Otherwise, it should return `null`. In the `TestCat` class, use the `createCat` method to create a `cat` object, and then try to print the value of the `name` field. What happens if you pass `null` as an argument to the `createCat` method? Analyze the result.

12. Perform the following steps in order:

    A. Create a class `Person` with the field `name`. Add to the class a method `introduceYourself` that displays the message "Hi, I'm" followed by the person's name. In the `TestPerson` class, create a `Person` object and call the `introduceYourself` method on it. Do you need to use the `this` keyword in the implementation of the method?

    B. Add to the `Person` class a method `sayHello` that takes another object of the `Person` class as an argument and displays the message "Hello," followed by the other person's name. Analyze how it works.

    C. Add to the `Person` class a method `changeName` that takes a string as an argument and assigns it to the `name` field. Create a `Person` object and use the `changeName` method to change its name. Then call the `introduceYourself` method. Has the name been changed? Do you need to use the `this` keyword in the implementation of the method?

    D. Add to the `Person` class a method `swapNames` that takes another object of the `Person` class as an argument and swaps the names of the two people. Create two `Person` objects and use the `swapNames` method to swap their names. Then call the `introduceYourself` method on both objects. Have the names been swapped?

13. Perform the following steps in order:

    A. Create a class `Counter` with one field, `number`. Add to the class a method `increase` that takes a parameter of type int and increases the value of the `number` field by the value of that parameter. In the `TestCounter` class, create a variable of type int, pass it to the `increase` method and check whether the value of the variable changed after calling the method.

B. Add to the `Counter` class a method `add` that takes another `Counter` object and adds the value of its `number` field to the `number` field of the current object. Create two `Counter` objects in the `TestCounter` class and use the `add` method. Check whether the object passed as the argument of the `add` method changed its value after calling the method.

C. Create a class `Modifier` that has a method `changeValue` which takes an int as an argument. Inside the method, change the value of the argument to a different one. In the `TestCounter` class, create an int variable, then pass it to the `changeValue` method and check whether the value of the variable has changed.

D. Add to the `Modifier` class a method `changeObject` that takes a `Counter` object as an argument. Inside the method, change the value of the `number` field of the `Counter` object to a different one. In the `TestCounter` class, create a `Counter` object, then pass it to the `changeObject` method and check whether the value of the object's `number` field has changed.

10 Access Modifiers

1. Create a class `Person` with a public field `name` and a private field `password`. See how different access modifiers affect access to these fields from another class.
2. Create two classes: `Parent` and `Child`. The `Parent` class should have one `protected` field. Try to access this field from the `Child` class.
3. Create a class `Car` with a private method `engineFailure()`. Try to call this method from outside the class.
4. Create two classes in the same package: `Employee` and `Company`. The `Employee` class should have a field without an access modifier. Try to access this field from the `Company` class.
5. Create a class `BankAccount` with a public field `accountNumber` and a private field `balance`. See how different access modifiers affect access to these fields from another class.
6. Create a class `Player` with three fields: `name` (public), `age` (with no access modifier) and `nationality` (private). In the `Main` class, create an array of players and try to change the values of all the fields. Observe the results.
7. Create a class `Coach` that is declared as `private`. Try to create a `Coach` object in the `Main` class. What is the result?

11 Constructor

1. Write a class `Book` that contains three fields: `title`, `author`, `publicationYear`. Then implement two constructors - a default one that takes no arguments, and a second one that takes three arguments corresponding to the fields of the class. In the case of the second constructor, the parameter names must be the same as the field names. Check whether you are able to correctly assign values to the fields of the class using the `this` keyword.
2. Design a class `Person` that contains two fields: `firstName`, `lastName`. Implement a constructor that takes two arguments corresponding to the fields of the class. The parameter names must be the same as the field names. Try to assign values to the fields of the class without using the `this` keyword. Is it possible? If not, why not?
3. Create a class `Car` that contains three fields: `brand`, `model`, `productionYear`. Implement three constructors - the first a default one, the second taking two arguments (`brand` and `model`), the third taking three arguments (`brand`, `model`, `productionYear`). In the case of the second and third constructors, the parameter names must be the same as the field names. Use the `this` keyword to distinguish the fields of the class from the parameters.
4. Write a class `Person` that has the fields `firstName`, `lastName` and `age`. Define two constructors in it, one taking all three parameters and the other only the first name and last name. The second constructor should call the first one, passing it a default age value of 0.
5. Design a class `Square` that has a field `side` and a constructor that allows setting the value of this field. Then write a second constructor that takes no arguments and only calls the first constructor with a default value of 1.
6. Define a class `Car` that has the fields `brand`, `model` and `productionYear`. The class should contain two constructors: the first taking all three parameters, and the second only the brand and model. The second constructor should call the first one, passing it the current year as the default value of `productionYear`.
7. Create a class `Dog` that has the fields `name`, `breed` and `age`. The class should have two constructors: one that takes all three parameters, and a second one that takes only the name and breed. The second constructor should call the first one, passing it a default age value of 1.

8. Write a class `Student` that has the fields `firstName`, `lastName` and `fieldOfStudy`. Define two constructors in it, one taking all three parameters and the other only `firstName` and `lastName`. The second constructor should call the first one, passing it `unknown` as the default value of `fieldOfStudy`.

9. Define a class `Employee` that has the fields `firstName`, `lastName` and `salary`. Write a constructor that takes two arguments (`firstName` and `lastName`), and an initializer block that sets the value of `salary` to 3000. Check what happens when you try to overwrite the value of `salary` in the constructor.

10. Design a class `Car` that has the fields `brand`, `model` and `price`. Write a constructor that takes two arguments (`brand` and `model`), and in an initializer block set the default value of `price` to 50000. See what happens when you try to change the value of `price` in the constructor.

11. Define a class `Phone` that has the fields `manufacturer`, `model` and `operatingSystem`. In an initializer block, set the value of `operatingSystem` to `Android`. Then write a constructor that takes two arguments (`manufacturer` and `model`) and tries to overwrite the value of operatingSystem with `iOS`. Check which value is ultimately assigned to the operatingSystem field.

12. Create a class `Car` containing the private fields `brand`, `model`, `productionYear`, `mileage` and `color`. Add a constructor that takes all the fields as arguments. Add accessor methods (getters and setters) for all the fields. Then add a method `displayInformation()` that displays all the information about the car.

13. Create a class `Person` with the private fields `firstName`, `lastName`, `age`, `address`. Add a constructor that takes all the fields as arguments. Add accessor methods (getters and setters) and a method `introduceYourself()` that returns a string with information about the person.

14. Create a class `Book` with the private fields `title`, `author`, `publicationYear`, `publisher` and `numberOfPages`. Add a constructor that takes all the fields as arguments. Add accessor methods (getters and setters) and a method `showInformation()` that displays information about the book.

15. Create a class `Point2D` with the private fields `x` and `y`, representing the coordinates of a point on a plane. Add a constructor that takes the coordinates as arguments. Add accessor methods (getters and setters) and a method `distance(Point2D anotherPoint)` that calculates the distance between two points on the plane.

16. Create a class `Rectangle` with the private fields `width` and `height`. Add a constructor that takes the side lengths as arguments. Add accessor methods (getters and setters) and the methods `area()` and `perimeter()`, which calculate the area and perimeter of the rectangle.

17. Create a class `Circle` with the private field `radius`. Add a constructor that takes the radius as an argument. Add accessor methods (getters and setters) and the methods `area()` and `circumference()`, which calculate the area and circumference of the circle.

18. Create a class `Student` with the private fields `firstName`, `lastName`, `indexNumber`, `yearOfStudy` and `gradeAverage`. Add a constructor that takes all the fields as arguments. Add accessor methods (getters and setters) and a method `showInformation()` that displays information about the student.

19. Create a class `Employee` with the private fields `firstName`, `lastName`, `position`, `age` and `salary`. Add a constructor that takes all the fields as arguments. Add accessor methods (getters and setters) and a method `showInformation()` that displays information about the employee.

20. Create a class `BankAccount` with the private fields `accountNumber`, `owner`, `balance` and `accountType`. Add a constructor that takes all the fields as arguments. Add accessor methods (getters and setters) and the methods `deposit(double amount)` and `withdraw(double amount)`, which respectively add or subtract an amount from the account balance.

21. Create a class `Television` with the private fields `brand`, `screenDiagonal`, `resolution`, `isSmartTV` and `price`. Add a constructor that takes all the fields as arguments. Add accessor methods (getters and setters) and a method `showInformation()` that displays information about the television.

22. Create a class `Gradebook` with the private fields `firstName`, `lastName` and `grades` (as an `ArrayList` of type `int`). Add a constructor that takes firstName and lastName as arguments. Add accessor methods (getters and setters) and the methods `addGrade(int grade)` and `removeGrade(int index)`, which respectively add or remove a grade from the list of grades. Also add a method `averageGrade()` to calculate and return the average of the grades.

23. Create a class `TemperatureHistory` with the private field `temperatures` (as an `ArrayList` of type `double`). Add a default constructor. Add accessor methods (getters and setters) and the methods `addTemperature(double temperature)` and `removeTemperature(int index)`, which respectively add or remove a temperature from the list of temperatures. Also add a method `averageTemperature()` to calculate and return the average of the temperatures.

24. Create a class `Results` (test results) with the private fields `firstName`, `lastName` and `results` (as an array of type `int`). Add a constructor that takes `firstName`, `lastName` and the array size as arguments. Add accessor methods (getters and setters) and a method `addResult(int index, int result)` that adds a test result at the given index. Also add a method `averageResult()` to calculate and return the average result.

25. Create a class `TaskManager` with the private field `taskPriorities` (as an `ArrayList` of type `int`). Add a default constructor. Add accessor methods (getters and setters) and the methods `addPriority(int priority)` and `removePriority(int index)`, which respectively add or remove a priority from the list of priorities. Also add a method `highestPriority()` to find and return the value of the highest priority.

26. Create a class `Warehouse` with the private field `productQuantities` (as an array of type `int`). Add a constructor that takes the array size as an argument. Add accessor methods (getters and setters) and a method `addProducts(int index, int quantity)` that adds the specified quantity of products at the given index. Also add a method `totalProducts()` to calculate and return the total of all products in the warehouse.

27. Define a class `Person` that has the fields `firstName`, `lastName` and `age`. Write a constructor that takes three arguments and validates them before assigning them to the corresponding fields. The person's age (`age`) should not be negative, and `firstName` and `lastName` should not be empty or null (for invalid values, set an empty string or zero individually).

28. Create a class `Point` representing a point in 3D space, with the fields `x`, `y`, `z`. Write a constructor that takes these three values and checks whether they are in the range from -100 to 100. If the values are not in this range, they should be set to the nearest bound.

29. Design a class `Car` that has the fields `brand`, `model` and `productionYear`. Write a constructor that takes three arguments. The constructor's task is to check whether `productionYear` is not greater than the current year and whether `brand` and `model` are not empty or null (for invalid values, set an empty string or the current year individually).

Hint: use `Calendar.getInstance().get(Calendar.YEAR)` to get the current year.

30. Define a class `BankAccount` that has the fields `accountNumber` and `balance`. Write a constructor that takes two arguments. Before assigning a value to the `accountNumber` field, check whether it has 26 digits, and before assigning a value to the `balance` field, check whether `balance` is not negative (for invalid values, set the account number so that it consists only of ones and the balance to zero).

12 Static Fields and Methods

1. Create a class `Person`. The class should contain a field `name` and a static field `counter`. The task is to increase the value of `counter` every time a new instance of the `Person` class is created. Add a method that returns the value of `counter`. Create a test case.
2. Create a class `Mathematics` that has a static field `PI` storing an approximate value of π. Make sure that the value of this field cannot be modified. Create a test case.
3. Create a class `Singleton` that uses a static field to store one and only one instance of this class. The class should contain a private constructor and a public static method `getInstance()` that returns the single instance of the `Singleton` class. Create a test case.
4. Create a class `Configuration` that contains a static field `applicationVersion` with a default value of "1.0". Add methods for reading and changing this value. Create a test case.
5. Create a class `Calculator` that has basic mathematical methods: addition, subtraction, multiplication and division. Implement these methods as static methods. Create a second class in which you test these methods without creating an instance of the `Calculator` class.
6. Create a class `ShapeFactory` that has static methods for creating various geometric shapes, such as circles, squares or triangles. Then create a test class in which you use these methods to create various shapes without the need to create an instance of the `ShapeFactory` class.
7. Create a class `Settings` that has various configuration parameters of an application (e.g. version, language). Add non-static methods for changing these parameters and a static method `defaultSettings()` that returns the default configuration of the application. In a test class, see how both groups of methods work and what the differences between them are.
8. Create a class `Configuration` that has a static field `applicationVersion` and a static field `applicationName`. Mark one of these fields as `final`. Consider what would happen if you wanted to update the value of one of these fields while the application is running.

9. Create a class `Product` that contains a static field `numberOfProducts` and a static field `MAX_PRODUCTS`. The `numberOfProducts` field will be used to count the number of products created, and `MAX_PRODUCTS` to limit their number. Mark only one of these fields with the `final` keyword and consider the consequences of this choice.

10. In a single project, do the following:

A. Create a class `SportsFacility`, which should be placed in the package `pl.edu.uwm.wmii.sports`.

B. The class should contain three attributes:

- `type` (type of facility), of type `String`.
- `location`, of type `String`.
- `capacity`, of type `int` (in number of people).

C. In the `SportsFacility` class, implement a static method `createStadium(String type, String location, int capacity)`. The method should return a new object of type `SportsFacility` whose fields are set from the method's arguments.

D. In the `SportsFacility` class, implement a non-static method `createFacility(String type, String location, int capacity)`. The method should return a new object of type `SportsFacility` whose fields are set from the method's arguments.

E. Create a class `TestSportsFacility` and place it in another file in the package `pl.edu.uwm.wmii.sports`. Add a `main` method to the `TestSportsFacility` class. In it, call the methods from points C and D.

11. In a single project, do the following:

A. Create a class `Software`, which should be placed in the package `pl.edu.uwm.wmii.software`.

B. The class should contain three attributes:

- `type` (type of software), of type `String`.
- `version`, of type `String`.
- `licenseCount` (number of licenses), of type `int`.

C. In the `Software` class, implement a static method `isLicenseAvailable` with an argument of type `Software`. The method should return a boolean value indicating whether the number of licenses of the software passed as the argument is greater than 3. For an argument that is `null`, return `false`.

D. In the `Software` class, implement a non-static method `isLicenseAvailable` with no argument. The method should return a boolean value indicating whether the number of licenses of the current object is greater than 3.

E. Create a class `TestSoftware` and place it in another file in the package `pl.edu.uwm.wmii.software`. Add a `main` method to the `TestSoftware` class. In it, call the methods from points C and D.

13 Records

1. Create a record `BookDTO` that represents a book in an online store. It should contain information such as `title`, `author`, `price` and `yearOfPublication`. Then create several instances of this record representing different books.
2. Create a record `Address` that contains `street`, `houseNumber`, `postalCode` and `city`. Then create a record `Person` that, in addition to basic information about a person (e.g. `firstName`, `lastName`), also contains a field of type `Address`. Create a test case.
3. Create a class `PointC` representing a point on a 2D plane with the fields `x` and `y`. Then create a record `PointR` with the same fields. Add methods enabling conversion from a class instance to a record and vice versa. Create a test case.
4. Create a record `Book` that represents `title`, `author` and `yearOfPublication`. Add a method `describe()` that returns the book information formatted in the style "Author - Title (Year of publication)". Create a test case.
5. Create a record `Point2D` representing a point on a 2D plane with the coordinates `x` and `y`. Introduce a method `distanceTo(Point2D otherPoint)` that calculates and returns the Euclidean distance between the current point and the one given as the argument. Create a test case.
6. Create a record `Car` that contains `brand`, `model` and `fuelConsumptionPer100km`. Add a method `fuelCost(double fuelPrice, double distance)` that calculates and returns the cost of a trip based on the given fuel price and distance. Create a test case.
7. Create a record `Student` that represents a student's first name, last name and list of grades. Introduce a method `averageGrades()` that calculates the student's grade average. Create a test case.
8. Create a record `Order` that contains a list of products and their prices. Each product is represented by a record `Product` with a name and a price. In the `Order` record, add a method `totalValue()` that calculates the total value of the order based on the product prices. Create a test case.
9. Create a record `Person` that represents a person's name and age. Add a constructor that verifies that the given age is not negative. If a negative value is given, set the age to 0. Create a test case.

10. Create a record `BankAccount` that contains an account number and a balance. Add a constructor that allows creating an account with just a number, with a default balance of 0. Create a test case.

11. Create a record `MusicTrack` that describes the title, artist and duration of a track in seconds. Add a constructor that takes only the title and artist, assuming a default track length of 180 seconds. Create a test case.

14 The `toString`, `equals` and `hashCode` Methods

1. Perform the following steps:

- Define a class `Person` that has the following fields: `firstName`, `lastName` and `age`.
- Write a constructor that takes three arguments and validates them before assigning them to the corresponding fields.
  - The person's age (`age`) should not be negative. If a negative value is given for the age, set the person's age to zero.
  - The `firstName` and `lastName` fields should not be empty or equal to null. If an empty string or null is given for these fields, set an empty string accordingly.
- Add a method `toString()` that returns information about the person in the format: `"Person: [firstName] [lastName], Age: [age]."`. Pay attention to letter case and punctuation.
- Add a method `equals()` that compares two people based on their `firstName`, `lastName` and `age` fields. Two people are considered identical if all three fields are the same.
- Add a method `hashCode()` that generates a hash code for the given object. This method should be consistent with the `equals()` method.

2. Perform the following steps:

- Define a class `Vehicle` that has the following fields: `brand`, `model` and `yearOfProduction`.
- Write a constructor that takes three arguments and validates them before assigning them to the corresponding fields.
  - The year of production (`yearOfProduction`) should not be greater than the current year. If a greater value is given, set the year of production to the current year.
  - The `brand` and `model` fields should not be empty or equal to null. If an empty string or null is given for these fields, set an empty string accordingly.
- Add a method `toString()` that returns information about the vehicle in the format: `"Vehicle: [brand] [model], Year: [yearOfProduction]."`.
- Add a method `equals()` that compares two vehicles based on their `brand`, `model` and `yearOfProduction` fields.

- Add a method `hashCode()` that generates a hash code for the given object. This method should be consistent with the `equals()` method.

3. Perform the following steps:

- Define a class `Book` that has the following fields: `title`, `author` and `numberOfPages`.
- Write a constructor that takes three arguments and validates them before assigning them to the corresponding fields.
  - The number of pages (`numberOfPages`) should not be negative. If a negative value is given, set the number of pages to one.
  - The `title` and `author` fields should not be empty or equal to null. If an empty string or null is given, set an empty string accordingly.
- Add a method `toString()` that returns information about the book in the format: `"Book: [title] by [author], Pages: [numberOfPages]."`.
- Add a method `equals()` that compares two books based on their `title`, `author` and `numberOfPages` fields.
- Add a method `hashCode()` that generates a hash code for the given object. This method should be consistent with the `equals()` method.

4. Perform the following steps:

- Define a class `Product` that has the following fields: `productName`, `category` and `price`.
- Write a constructor that takes three arguments and validates them before assigning them to the corresponding fields.
  - The price (`price`) should not be negative. If a negative value is given, set the product price to zero.
  - The `productName` and `category` fields should not be empty or equal to null. If an empty string or null is given, set an empty string accordingly.
- Add a method `toString()` that returns information about the product in the format: `"Product: [productName], Category: [category], Price: [price]."`.
- Add a method `equals()` that compares two products based on their `productName`, `category` and `price` fields.
- Add a method `hashCode()` that generates a hash code for the given object. This method should be consistent with the `equals()` method.

4. Perform the following steps:

- Create a class `Results` (test results) with the private fields `firstName`, `lastName` and `results` (as an array of type `int`). Add a constructor that takes `firstName`, `lastName` and the array size as arguments. Add accessor methods (getters and setters) and a method `addResult(int index, int result)` that adds a test result at the given index. Also add a method `averageResult()` to calculate and return the average result.

- Add a method `toString()` that returns information about the student, their results and the contents of the `results` array in the format: `"Results for [firstName] [lastName]: Average Score = [averageResult], Results: [result1, result2, ...]. "`. In addition, add a method `equals()` that compares two objects of the `Results` class based on their `firstName` and `lastName` fields and the contents of the `results` array. Two objects are considered identical if all fields and the test results in the arrays are the same. Also add a method `hashCode()` that generates a hash code for the given object. This method should be consistent with the `equals()` method.

5. Perform the following steps:

- Create a class `CDAlbum` with the private fields `artistName`, `albumTitle` and `tracks` (as an array of type `int` representing the length of each track in minutes).
- Add a constructor that takes `artistName`, `albumTitle` and the number of tracks as arguments.
- Add accessor methods (getters and setters) and a method `addTrack(int index, int length)` that adds the length of a track at the given index.
- Also add a method `totalLength()` to calculate and return the total length of all tracks.
- Add a method `toString()` that returns information about the artist, the album and the duration of all tracks in the format: `"CDAlbum by [artistName]: [albumTitle], Total Length = [totalLength] minutes, Tracks Length: [length1, length2, ...]. "`.
- Add a method `equals()` that compares two objects of the `CDAlbum` class based on their `artistName` and `albumTitle` fields and the contents of the `tracks` array. Two albums are considered identical if all the information is the same.
- Add a method `hashCode()` that generates a hash code for the given object. This method should be consistent with the `equals()` method.

6. Perform the following steps:

- Create a class `MovieRating` with the private fields `movieTitle`, `directorName` and `ratings` (as an array of type `double` representing the movie's ratings on a scale from 1 to 10).
- Add a constructor that takes `movieTitle`, `directorName` and the number of ratings as arguments.
- Add accessor methods (getters and setters) and a method `addRating(int index, double rating)` that adds a movie rating at the given index.
- Also add a method `averageRating()` to calculate and return the average rating of the movie.
- Add a method `toString()` that returns information about the movie, the director, the average rating and all ratings in the format: `"MovieRating for [movieTitle] by [directorName]: Average Rating = [averageRating], Ratings: [rating1, rating2, ...]. "`.

- Add a method `equals()` that compares two objects of the `MovieRating` class based on their `movieTitle` and `directorName` fields and the contents of the `ratings` array. Two movies are considered identical if all the information is the same.
- Add a method `hashCode()` that generates a hash code for the given object. This method should be consistent with the `equals()` method.

7. Perform the following steps:

- Create a class `Gradebook` with the private fields `firstName`, `lastName` and `grades` (as an `ArrayList` of type `int`). Add a constructor that takes firstName and lastName as arguments. Add accessor methods (getters and setters) and the methods `addGrade(int grade)` and `removeGrade(int index)`, which respectively add or remove a grade from the list of grades. Also add a method `averageGrade()` to calculate and return the average of the grades.
- Add a method `toString()` that returns information about the student, their grade average and all grades in the format: `"Gradebook for [firstName] [lastName]: Average Grade = [averageGrade], Grades: [grade1, grade2, ...]."`. Pay attention to letter case and punctuation.
- Add a method `equals()` that compares two objects of the `Gradebook` class based on their `firstName` and `lastName` fields and the contents of the `grades` list. Two gradebooks are considered identical if they have the same first name, last name and an identical set of grades (taking order into account).
- Add a method `hashCode()` that generates a hash code for the given object. This method should be consistent with the `equals()` method.

8. Perform the following steps:

- Create a class `Wallet` with the private fields `ownerName`, `ownerSurname` and `coins` (as an `ArrayList` of type `double`). Add a constructor that takes `ownerName` and `ownerSurname` as arguments. Add accessor methods (getters and setters) and the methods `addCoin(double coin)` and `removeCoin(int index)`, which respectively add or remove a coin from the list of coins.
- Also add a method `totalAmount()` to calculate and return the total amount in the wallet.
- Add a method `toString()` that returns information about the wallet owner, the total amount and all coins in the format: `"Wallet of [ownerName] [ownerSurname]: Total Amount = [totalAmount], Coins: [coin1, coin2, ...]."`. Pay attention to letter case and punctuation.
- Add a method `equals()` that compares two objects of the `Wallet` class based on their `ownerName` and `ownerSurname` fields and the contents of the `coins` list. Two wallets are considered identical if they have the same owner and an identical set of coins (regardless of order).

- Add a method `hashCode()` that generates a hash code for the given object. This method should be consistent with the `equals()` method.

9. Perform the following steps:

- Create a class `LibraryCard` with the private fields `holderFirstName`, `holderLastName` and `booksBorrowed` (as an `ArrayList` of type `String`). Add a constructor that takes `holderFirstName` and `holderLastName` as arguments. Add accessor methods (getters and setters) and the methods `borrowBook(String bookTitle)` and `returnBook(String bookTitle)`, which respectively add or remove a book title from the list of borrowed books.
- Also add a method `numberOfBooksBorrowed()` to calculate and return the number of borrowed books.
- Add a method `toString()` that returns information about the card holder, the number of borrowed books and all titles of the borrowed books in the format: `"Library card of [holderFirstName] [holderLastName]: Number of books borrowed = [numberOfBooksBorrowed], Books: [book1, book2, ...]."`. Pay attention to letter case and punctuation.
- Add a method `equals()` that compares two objects of the `LibraryCard` class based on their `holderFirstName` and `holderLastName` fields and the contents of the `booksBorrowed` list. Two library card holders are considered identical if they have the same first name, last name and an identical set of borrowed books (preserving order).
- Add a method `hashCode()` that generates a hash code for the given object. This method should be consistent with the `equals()` method.

15 Inheritance

1. Perform the following steps in order:

   A. Create a base class `Person` with a private field `firstName` and a protected field `lastName`. Then create a class `Employee` that inherits from the `Person` class. In the `Employee` class, try to refer to both fields and notice which of them are accessible.
   B. Based on the `Person` class from the previous subtask, create accessor methods (getters) for both fields. In the `Employee` class, create a method `displayData` that uses these accessor methods to print information about the employee. Consider why accessor methods are used to access private fields.

2. Create a base class `Book` with a private field `title` and a protected field `author`. Create a subclass `Ebook` that inherits from the `Book` class. In the `Ebook` class, try to change the access modifier of both fields from the base class. Consider why one of the fields allows this and the other does not.

3. Create a class `Tool` with a protected constructor that takes the name of the tool. Then create a subclass `Hammer` that inherits from the `Tool` class. In the `Hammer` class, create a constructor that uses the constructor of the base class. Consider why this scenario would not be possible if the `private` modifier were used for the base class constructor.

4. Create two different packages: `animals` and `mammals`. In the `animals` package, create a base class `Animal` with a protected field `species` and a private field `age`. In the `mammals` package, create a class `Dog` that inherits from the `Animal` class. Try to refer to both fields of the base class in the `Dog` class and notice which of them are accessible.

5. Perform the following steps in order:

   A. Create a base class `Vehicle` with a method `drive` that prints "The vehicle is moving.". Then create a subclass `Car` that overrides the `drive` method so that it prints "The car is moving.". Create an object of the `Car` class and call its `drive` method to observe the result.
   B. Based on the `Vehicle` class from the previous subtask, in the `Car` class, override the `drive` method so that it calls the original method from the base class and additionally prints information specific to the `Car` class. Use the `super` keyword to call the method from the base class.

6. Create a class `Calculator` with a method `add` that takes two arguments of type `int` and returns their sum. Then create a subclass `ExtendedCalculator` that overrides the `add` method in such a way that it takes three arguments of type `int` and returns their sum. Consider whether this is actually method overriding or perhaps something else.

7. Create a base class `Base` with a static method `info` that prints "This is the base class.". Then create a subclass `Child` and try to override the static method `info` so that it prints "This is the child class.". Consider the behavior of methods "overridden" this way and why such hiding is treated differently.

8. Create a base class `Computer` with a protected method `start` that prints "Computer started.". Create a subclass `Laptop` that tries to override the `start` method, but with the `public` access modifier. Try to compile the code and consider why there are certain restrictions on overriding methods with respect to access modifiers.

9. Create a base class named `Vehicle` with the fields `brand` and `model`. This class should have a constructor that takes both of these parameters. Then create a subclass named `Car` that inherits from the `Vehicle` class. The `Car` class should have an additional field `numberOfDoors`. Create a constructor for the `Car` class that takes all three parameters and uses the constructor of the base class. Create a test case.

10. Create a base class named `Animal` with a field `name` and a constructor that assigns the default name "Unknown". Then create a class `Dog` that inherits from the `Animal` class. Do not create an additional constructor in it. Check what animal is created when you create a new instance of the `Dog` class.

11. Create a base class named `Building` with the fields `height` and `color`. This class should have two constructors: one taking both parameters and a second, parameterless one that assigns default values. Create a subclass `House` with an additional field `numberOfRooms`. The `House` class should have a constructor that uses an overloaded constructor of the base class. Create a test case.

12. Create three classes: `Entity`, `Human` and `Programmer`. The `Human` class should inherit from the `Entity` class, and the `Programmer` class from the `Human` class. In each class, add a constructor that prints information about the creation of an instance of that class. Create an instance of the `Programmer` class and observe the order in which the constructors are called.

13. Create a class `Vehicle` with the fields `brand`, `model` and `yearOfProduction`. Create classes `Car` and `Motorcycle` that inherit from the `Vehicle` class. The `Car` class should have an additional field `numberOfDoors`, and the `Motorcycle` class a field `engineCapacity`. Add constructors, getter and setter methods, and the methods `toString()`, `equals()` and `hashCode()` for each of the classes. Write a program that tests the defined classes and methods.

14. Create a class `Employee` with the fields `firstName`, `lastName` and `salary`. Create classes `Programmer` and `Tester` that inherit from the `Employee` class. The `Programmer` class should have an additional field `programmingLanguage`, and the `Tester` class a field `testingType`. Add constructors, getter and setter methods, and the methods `toString()`, `equals()` and `hashCode()` for each of the classes. Write a program that tests the defined classes and methods.

15. Create a class `Property` with the fields `address`, `size` and `price`. Create classes `House` and `Apartment` that inherit from the `Property` class. The `House` class should have an additional field `numberOfFloors`, and the `Apartment` class a field `floorNumber`. Add constructors, getter and setter methods, and the methods `toString()`, `equals()` and `hashCode()` for each of the classes. Write a program that tests the defined classes and methods.

16. Create a class `BoardGame` with the fields `gameName`, `minPlayers`, `maxPlayers` and `rules` (as an ArrayList of type `String`). Create classes `EducationalGame` and `StrategicGame` that inherit from the `BoardGame` class. The `EducationalGame` class should have an additional field `subject`, and the `StrategicGame` class a field `duration`. Add constructors, getter and setter methods, and the methods `toString()`, `equals()` and `hashCode()` for each of the classes. Write a program that tests the defined classes and methods.

17. Create a class `Team` with the fields `name`, `city` and `points` (as an ArrayList of type `Integer`). Create classes `SoccerTeam` and `VolleyballTeam` that inherit from the `Team` class. The `SoccerTeam` class should have an additional field `rankingPosition`, and the `VolleyballTeam` class a field `numberOfVictories`. Add constructors, getter and setter methods, and the methods `toString()`, `equals()` and `hashCode()` for each of the classes. Write a program that tests the defined classes and methods.

18. Create a class `Computer` with the fields `manufacturer`, `model` and `partsPrices` (as an ArrayList of type `Double`). Create classes `Laptop` and `Desktop` that inherit from the `Computer` class. The `Laptop` class should have an additional field `weight`, and the `Desktop` class a field `caseType`. Add constructors, getter and setter methods, and the methods `toString()`, `equals()` and `hashCode()` for each of the classes. Write a program that tests the defined classes and methods.

19. Create a class `MusicAlbum` with the fields `title`, `artist` and `ratings` (as an ArrayList of type `Integer`). Create classes `RockAlbum` and `JazzAlbum` that inherit from the `MusicAlbum` class. The `RockAlbum` class should have an additional field `rockGenre`, and the `JazzAlbum` class a field `jazzGenre`. Add constructors, getter and setter methods, and the methods `toString()`, `equals()` and `hashCode()` for each of the classes. Write a program that tests the defined classes and methods.

20. Create a class `Book` with the fields `author`, `title` and `yearOfPublication`. Then create two derived classes: `Ebook`, which adds a field `fileSize`, and `PaperbackBook`, which adds a field `numberOfPages`. Each of the classes should contain constructors, getter and setter methods, and the methods `toString()`, `equals()` and `hashCode()`. In the `equals()`

method of the `Book` class compare only `author`, in the `Ebook` class compare by `author` and `fileSize`, and in the `PaperbackBook` class compare `author` and `numberOfPages`. Write a program that tests these classes, demonstrating polymorphism and object comparison.

21. Design a class `Employee` with the fields `name`, `department` and `salary`. Create two derived classes: `Manager` with an additional field `bonus` and `Intern` with a field `internshipLength`. Each class should have constructors, getters and setters, `toString()`, `equals()` and `hashCode()`. In the `equals()` method of the `Employee` class compare only `name`, in the `Manager` class compare by `name` and `bonus`, and in the `Intern` class compare `name` and `internshipLength`. Write a program that tests these classes, demonstrating polymorphism and object comparison.

16 Packages

1. Create a package named `cars`. Inside this package, create a class `Car` containing three private fields: `brand`, `model` and `yearOfProduction`. Add the appropriate accessor methods (getters and setters) to the class and a constructor that takes all three fields. In a test class outside this package, create an object of the `Car` class, assign values to it and print them to the screen.

2. Create a package named `animals`. Inside this package, create two classes: `Dog` and `Cat`. Both should contain the fields `name` and `age`. Each of the classes should have a method `makeSound()`. For the `Dog` class this method should print "Woof, woof!", and for the `Cat` class - "Meow!". In a test class outside this package, create objects of both classes, assign values to them and call their `makeSound()` methods.

3. Create two packages: `employees` and `tools`. In the `employees` package, create a class `Employee` with the fields `firstName`, `lastName` and `salary`. The `salary` field should be protected. In the `tools` package, create a class `SalaryCalculator` that has a method `raiseSalary(Employee employee, double percent)` which increases the employee's salary by the given percentage. Try to call this method in a test class outside both packages, and consider the problems of accessing protected fields across different packages.

4. Create a package named `electronics`. Inside this package, create two subpackages: `televisions` and `phones`. In the `televisions` subpackage, create a class `Television` with the fields `brand` and `screenDiagonal`. In the `phones` subpackage, create a class `Phone` with the fields `brand` and `operatingSystem`. For both classes, add the appropriate getters, setters and constructors. In a test class outside the `electronics` package, create objects of both classes, assign values to them and display them.

5. Create two packages: `books` and `library`. In the `books` package, create a class `Book` with the fields `title`, `author` and `publicationYear`. In the `library` package, create a class `Shelf` containing a list of books and methods for adding and removing books. To use the `Book` class in the `library` package, you must import the appropriate package. In a test class, create several books, add them to the shelf and print the contents of the shelf.

6. Create a package `bank`. Inside this package, create two classes: `Account` and `Bank`. The `Account` class should have the private fields `accountNumber`, `balance` and the methods `deposit(double amount)` and `withdraw(double amount)`. The `Bank` class should contain a list of accounts and methods for creating new accounts and carrying out

transfers between them. Try to create an account directly in a test class outside the `bank` package and consider how access modifiers affected access to the classes and methods in the package.

7. Create a package named `animals`. Inside this package, create a base class `Animal` with the fields `name` and `age` and a method `makeSound()`. Then, in the same package, create two derived classes: `Dog` and `Cat`, which inherit from the `Animal` class. The derived classes should override the `makeSound()` method. In a test class outside the `animals` package, create objects of both derived classes, assign values to them and call the `makeSound()` method for each of them.

8. Create a package named `computers`. In this package, create a base class `Computer` with the fields `brand` and `processor`. Then create two derived classes: `Laptop` and `Desktop`. The `Laptop` class should additionally have a field `weight`, and the `Desktop` class a field `caseType`. All classes should have the appropriate getters, setters and constructors. In a test class outside the `computers` package, create objects of both derived classes, assign values to them and print them.

9. Create a package named `company`. Inside this package, create a base class `Employee` with the fields `firstName`, `lastName` and `salary`. Then create two derived classes: `Manager` and `Developer`. The `Manager` class should additionally have a field `bonus`, and the `Developer` class a field `programmingLanguage`. Add appropriate methods for calculating annual earnings (salary + any bonuses). In a test class outside the `company` package, create objects of both derived classes, assign values to them and calculate their annual earnings.

17 Complex Fields in a Class

1. Create a class `MusicAlbum` with the fields `title`, `artist` and `ratings` (as an array with elements of type double). Add a method that allows adding and removing ratings. Create a class `RockAlbum` that inherits from the `MusicAlbum` class. The `RockAlbum` class should have an additional field `rockGenre`. Add constructors, getter and setter methods, and the methods `toString()`, `equals()` and `hashCode()` for each of the classes. Write a program that tests the defined classes and methods.

2. Create a class `Book` with the fields `title`, `author` and `reviews` (as an array with elements of type double). Add methods that allow adding and removing reviews. Create a class `FantasyBook` that inherits from the `Book` class. The `FantasyBook` class should have an additional field `fantasySubgenre`. Add constructors, getter and setter methods, and the methods `toString()`, `equals()` and `hashCode()` for each of the classes. Write a program that tests the defined classes and methods.

3. Create a class `ComputerGame` with the fields `title`, `producer` and `ratings` (as an array with elements of type double). Add methods that allow adding and removing ratings. Create a class `RPGGame` that inherits from the `ComputerGame` class. The `RPGGame` class should have an additional field `gameWorld`. Add constructors, getter and setter methods, and the methods `toString()`, `equals()` and `hashCode()` for each of the classes. Write a program that tests the defined classes and methods.

4. Create a class `University` with the fields `name`, `location` and `studyPrograms` (as an array with elements of type `String`). Add methods that allow adding and removing study programs. Create a class `TechnicalUniversity` that inherits from the `University` class. The `TechnicalUniversity` class should have an additional field `numberOfLaboratories`. Add constructors, getter and setter methods, and the methods `toString()`, `equals()` and `hashCode()` for each of the classes. Write a program that tests the defined classes and methods.

5. Create a class `ArtGallery` with the fields `name`, `city` and `paintings` (as an array with elements of type `String`). Add methods that allow adding and removing paintings. Create a class `ContemporaryGallery` that inherits from the `ArtGallery` class. The `ContemporaryGallery` class should have an additional field `numberOfInstallations`. Add constructors, getter and setter methods, and the methods `toString()`, `equals()` and `hashCode()` for each of the classes. Write a program that tests the defined classes and methods.

6. Create a class `Car` with the fields `brand`, `model` and `engineVariants` (as an array with elements of type `String`). Add methods that allow adding and removing engine variants. Create a class `ElectricCar` that inherits from the `Car` class. The `ElectricCar` class should have an additional field `range`. Add constructors, getter and setter methods, and the methods `toString()`, `equals()` and `hashCode()` for each of the classes. Write a program that tests the defined classes and methods.

7. Perform the following steps:

- Create a class `Engine` with the fields `power`, `type` (engine type) and `serialNumber`. Add a parameterized constructor, getters, setters, and the methods `toString`, `equals` and `hashCode`.
- Create a class `Car`. The `Car` class should have the fields `make`, `model` and `engine` of type `Engine`. Add a parameterized constructor that takes an `Engine` object as a parameter, getters, setters, and the methods `toString`, `equals` and `hashCode`.

8. Perform the following steps:

- Create a class `Processor` with the fields `frequency`, `cores` (number of cores) and `manufacturer`. Add a parameterized constructor, getters, setters, and the methods `toString`, `equals` and `hashCode`.
- Create a class `Computer`. The `Computer` class should have the fields `brand`, `model` and `processor` of type `Processor`. Add a parameterized constructor that takes a `Processor` object as a parameter, getters, setters, and the methods `toString`, `equals` and `hashCode`.

9. Perform the following steps:

- Create a class `Address` with the fields `street`, `city`, `zipCode` and `country`. Add a parameterized constructor, getters, setters, and the methods `toString`, `equals` and `hashCode`.
- Create a class `UserAccount`. The `UserAccount` class should have the fields `username`, `email` and `address` of type `Address`. Add a parameterized constructor that takes an `Address` object as a parameter, getters, setters, and the methods `toString`, `equals` and `hashCode`.

18 Abstract Classes

1. Define an abstract class `WorkTool` with the fields `name` of type `String` and `productionYear` of type `int`. Add an abstract method `use()` that will simulate using the tool. Then define the classes `Hammer`, `Screwdriver` and `Saw`, which inherit from the `WorkTool` class and implement the `use()` method. Create an array list of 5 appropriate objects and call the written method for them.
2. Define an abstract class `ComputerGraphic` with the fields `width`, `height` of type `int` and `fileName` of type `String`. Add the abstract methods `loadFile()` and `saveFile()`. Then define the classes `Bitmap` and `Vector`, which inherit from the `ComputerGraphic` class and implement the `loadFile()` and `saveFile()` methods. Create an array list of 5 appropriate objects and call the written methods for them.
3. Define an abstract class `ElectronicDevice` with the fields `manufacturer` of type `String`, `model` of type `String` and `productionYear` of type `int`. Add the abstract methods `turnOn()` and `turnOff()`. Then define the classes `Smartphone`, `Television` and `Laptop`, which inherit from the `ElectronicDevice` class and implement the `turnOn()` and `turnOff()` methods. Create an array list of 5 appropriate objects and call the written methods for them.
4. In a single project, do the following:

A. Create an abstract class `Product` containing a public abstract method `getPrice()` that takes no arguments and returns `double`. Place the class in the `store` package.

B. Create two classes derived from `Product`: `Book` and `Clothing`. In both classes, override the `getPrice()` method. For `Book` let it return the price 29.99, and for `Clothing` let it return the price 59.99.

C. In the `TestProduct` class in the `store` package, create an array of type `Product` and initialize it with 5 instances of `Book` and `Clothing`. Iterate over the array, calling the `getPrice()` method for each product (display the prices on standard output).

5. In a single project, do the following:

A. Create an abstract class `Game` containing a public abstract method `getRating()` that takes no arguments and returns `double`. Place the class in the `entertainment` package.

B. Create two classes derived from `Game`: `StrategyGame` and `AdventureGame`. In both classes, override the `getRating()` method. For `StrategyGame` let it return the rating 8.5, and for `AdventureGame` let it return the rating 7.3.

C. In the `TestGame` class in the `entertainment` package, create an array list of type `Game` and initialize it with 5 instances of `StrategyGame` and `AdventureGame`. Iterate over the array list, calling the `getRating()` method for each game (display the ratings on standard output).

19 Final Fields, Methods and Classes

1. Perform the following steps in order:

   A. Create a class `Planet` with one final field `name`. Try to change the value of the `name` field after it has been initialized. Analyze the result.

   B. Add to the `Planet` class a method `changeName` that tries to change the name of the planet. What is the result of trying to change a final field using a method?

2. Create a class `Car` with two fields: `brand` (a regular field) and `VIN` (a final field - Vehicle Identification Number). Create several `Car` objects and try to change the `VIN` field for each of them. Analyze the result.

3. Create a class `Athlete` with one final field `name` and a field `points`. Create an array of athletes and try to change the `name` field for one of the athletes. Analyze the result.

4. Create a class `Person` with one final field `nationalId` (national identification number). Then create a class `Student` that inherits from the `Person` class. Can you change the `nationalId` field in the `Student` class? What happens if you try to do it? Analyze the results.

5. Create a base class `Electronics` with a final method `turnOn` that prints "Device turned on". Then create a subclass `Television` and try to override the `turnOn` method. Observe what happens.

6. Create a class `Computer` with a final method `boot` and a regular method `launchApplication`. In the subclass `Laptop`, try to override both methods. Check which of the methods can be overridden and which cannot.

7. Create a final class `ImmutableData` with some attributes and methods. Then try to create a subclass `VariableData` that inherits from the `ImmutableData` class. Consider why you cannot inherit from a class marked as `final`.

8. Create a class `Game` whose constructor calls a final method `initialize` (which initializes some data). Create a subclass `RPG` that tries to customize the constructor of the base class. Make sure that the `initialize` method works correctly even though it is marked as `final`.

9. Create a class `MathConstant` with a final field `PI` that is initialized with the value 3.14159. In the subclass `PhysicalConstant`, try to create a method that tries to change the value of `PI`. Consider why the value of a final field cannot be changed after it has been initialized.

10. Perform the following steps:

- Create a class `ImmutableDate` with the private final fields `year`, `month`, `day`.
- Add a parameterized constructor to initialize all the fields.
- Add the public methods `getYear`, `getMonth`, `getDay` to retrieve the values of the fields, but do not add any methods that allow modifying them.
- Implement the methods `toString`, `equals` and `hashCode`.
- Check whether such a class can be considered immutable.

11. Perform the following steps:

- Create a class `ImmutablePoint` with the private final fields `x`, `y`, `z` (the coordinates of the point).
- Add a parameterized constructor to initialize all the fields.
- Add the public methods `getX`, `getY`, `getZ` to retrieve the values of the fields, but do not add any methods that allow modifying them.
- Implement the methods `toString`, `equals` and `hashCode`.
- Check whether such a class can be considered immutable.

12. Perform the following steps:

- Create a class `ImmutableBook` with the private final fields `title`, `author`, `isbn` (ISBN number).
- Add a parameterized constructor to initialize all the fields.
- Add the public methods `getTitle`, `getAuthor`, `getIsbn` to retrieve the values of the fields, but do not add any methods that allow modifying them.
- Implement the methods `toString`, `equals` and `hashCode`.
- Check whether such a class can be considered immutable.

20 The `Comparable` Interface

1. Write a class `Student` that contains the fields `name` (of type `String`), `averageGrade` (of type `double`) and `yearOfBirth` (of type `int`). Implement the `Comparable` interface in such a way that objects of the `Student` class are sorted in descending order by grade average. Create an array list of 5 objects of the `Student` class and sort it according to the specified criterion.

2. Write a class `Employee` that contains the fields `name` (of type `String`), `salary` (of type `double`) and `employmentDate` (of type `LocalDate`). Implement the `Comparable` interface in such a way that objects of the `Employee` class are sorted in ascending order by salary. Create an array list of 5 objects of the `Employee` class and sort it according to the specified criterion.

3. Write a class `Client` that contains the fields `name` (of type `String`), `clientNumber` (of type `int`) and `lastLogin` (of type `Date`). Implement the `Comparable` interface in such a way that objects of the `Client` class are sorted in descending order by the date of last login. Create an array list of 5 objects of the `Client` class and sort it according to the specified criterion.

4. Write a class `Product` that contains the fields `name` (of type `String`), `price` (of type `double`) and `productionDate` (of type `LocalDate`). Implement the `Comparable` interface in such a way that objects of the `Product` class are sorted in ascending order by production date. Create an array list of 5 objects of the `Product` class and sort it according to the specified criterion.

5. Write a class `Person` that contains the fields `name` (of type `String`), `height` (of type `int`) and `dateOfBirth` (of type `LocalDate`). Implement the `Comparable` interface in such a way that objects of the `Person` class are sorted in descending order by height. Create an array list of 5 objects of the `Person` class and sort it according to the specified criterion.

6. Write a class `Book` that contains the fields `title` (of type `String`), `numberOfPages` (of type `int`) and `publicationDate` (of type `LocalDate`). Implement the `Comparable` interface in such a way that objects of the `Book` class are sorted in descending order by number of pages. Create an array of 4 objects of the `Book` class and sort it according to the specified criterion.

7. Write a class `Car` that contains the fields `brand` (of type `String`), `mileage` (of type `int`) and `yearOfProduction` (of type `int`). Implement the `Comparable` interface in such a

way that objects of the `Car` class are sorted in ascending order by mileage. Create an array of 4 objects of the `Car` class and sort it according to the specified criterion.

8. Write a class `FoodProduct` that contains the fields `name` (of type `String`), `price` (of type `double`) and `expirationDate` (of type `LocalDate`). Implement the `Comparable` interface in such a way that objects of the `FoodProduct` class are sorted in ascending order by expiration date. Create an array of 4 objects of the `FoodProduct` class and sort it according to the specified criterion.

9. Write a class `Music` that contains the fields `title` (of type `String`), `artist` (of type `String`) and `releaseYear` (of type `int`). Implement the `Comparable` interface in such a way that objects of the `Music` class are sorted in descending order by release year. Create an array of 4 objects of the `Music` class and sort it according to the specified criterion.

10. Write a class `Item` that contains the fields `name` (of type `String`), `weight` (of type `double`) and `price` (of type `double`). Implement the `Comparable` interface in such a way that objects of the `Item` class are sorted in ascending order by weight. Create an array of 4 objects of the `Item` class and sort it according to the specified criterion.

11. Write a class `Student` that contains the fields `name` (of type `String`), `averageGrade` (of type `double`) and `yearOfStudy` (of type `int`). Implement the `Comparable` interface in such a way that objects of the `Student` class are sorted according to a single criterion: in descending order by grade average, and in case of a tie in ascending order by year of study. Create an array of 4 objects of the `Student` class and sort it according to the specified criterion.

12. Write a class `Order` that contains the fields `productName` (of type `String`), `quantity` (of type `int`) and `unitPrice` (of type `double`). Implement the `Comparable` interface in such a way that objects of the `Order` class are sorted according to a single criterion: in descending order by unit price, and in case of a tie in ascending order by quantity. Create an array list of 4 objects of the `Order` class and sort it according to the specified criterion.

13. Write a class `Client` that contains the fields `name` (of type `String`), `balance` (of type `double`) and `lastPurchaseDate` (of type `LocalDate`). Implement the `Comparable` interface in such a way that objects of the `Client` class are sorted according to a single criterion: in descending order by balance, and in case of a tie in ascending order by the date of the last purchase. Create an array list of 4 objects of the `Client` class and sort it according to the specified criterion.

14. Write a class `Course` that contains the fields `name` (of type `String`), `numberOfHours` (of type `int`) and `price` (of type `double`). Implement the `Comparable` interface in such a way that objects of the `Course` class are sorted according to a single criterion: in ascending order by number of hours, and in case of a tie in descending order by price. Create an array of 4 objects of the `Course` class and sort it according to the specified criterion.

15. Write a class `Product` that contains the fields `name` (of type `String`), `price` (of type `double`) and `expirationDate` (of type `LocalDate`). Implement the `Comparable` interface in such a way that objects of the `Product` class are sorted according to a single criterion: in descending order by expiration date, and in case of a tie in ascending order by price. Create a list of objects of the `Product` class and sort it according to the specified criterion. Then display the sorted list on the screen.

16. Write a class `Car` that contains the fields `brand` (of type `String`), `model` (of type `String`) and `registrationNumber` (of type `String`). Implement the `Comparable` interface in such a way that objects of the `Car` class are sorted according to a single criterion: in ascending order by the length of the registration number. Create an array of 4 objects of the `Car` class and sort it according to the specified criterion.

17. Write a class `Employee` that contains the fields `firstName` (of type `String`), `lastName` (of type `String`) and `position` (of type `String`). Implement the `Comparable` interface in such a way that objects of the `Employee` class are sorted according to a single criterion: in ascending order by the length of the last name. Create an array list of 4 objects of the `Employee` class and sort it according to the specified criterion.

18. Write a class `Movie` that contains the fields `title` (of type `String`), `director` (of type `String`) and `genre` (of type `String`). Implement the `Comparable` interface in such a way that objects of the `Movie` class are sorted according to a single criterion: in ascending order by the length of the title. Create an array list of 4 objects of the `Movie` class and sort it according to the specified criterion.

19. Write a class `Book` that contains the fields `title` (of type `String`), `author` (of type `String`) and `publishDate` (of type `LocalDate`). Implement the `Comparable` interface in such a way that objects of the `Book` class are sorted according to a single custom criterion: in ascending order by year of publication. Create an array of 4 objects of the `Book` class and sort it according to the specified criterion.

20. Write a class `Product` that contains the fields `name` (of type `String`), `price` (of type `double`) and `productionDate` (of type `LocalDate`). Implement the `Comparable` interface in such a way that objects of the `Product` class are sorted according to a single custom criterion: in descending order by year of production. Create an array list of 4 objects of the `Product` class and sort it according to the specified criterion.

21. Define a class `Client` that implements the generic `Comparable` interface. In this class, declare the private fields `lastName` of type `String` and `balance` of type `double`. When implementing the `compareTo` method of the `Comparable` interface, compare clients based on their balance, and in case of the same balance - based on last name. Then define a class `Company` that inherits from the `Client` class. The `Company` class should additionally have a private field `numberOfEmployees` of type `int`. When implementing the `compareTo` method of the `Comparable` interface in the `Company` class, use the `compareTo` method defined in the `Client` class and, if necessary, take the `numberOfEmployees` field into account. Write a program `TestClient` in which you create an array list of 5 clients and

companies named `clientList` using the `ArrayList` class. The list should include at least two clients with the same balance and different last names, and two companies with the same number of employees and different balances. Display the contents of `clientList`, sort it using the instance method `sort` of the `ArrayList` class, and display the contents of the list again.

22. Define a class `Animal` that implements the generic `Comparable` interface. In this class, declare the private fields `species` of type `String` and `age` of type `int`. When implementing the `compareTo` method of the `Comparable` interface, compare animals based on their age, and in case of the same age - based on species. Then define a class `Dog` that inherits from the `Animal` class. The `Dog` class should additionally have a private field `breed` of type `String`. When implementing the `compareTo` method of the `Comparable` interface in the `Dog` class, use the `compareTo` method defined in the `Animal` class and, if necessary, take the `breed` field into account. Write a program `TestAnimal` in which you create an array list of 5 animals and dogs named `animalList` using the `ArrayList` class. The list should include at least 3 objects of different types.

21 The `Comparator` Interface

1. Write a class `Person` with the fields `name` (`String`), `age` (`int`) and `height` (`double`). Write a class implementing the `Comparator` interface that compares people based on age. Create an array of 5 people and sort it by age.
2. Write a class `Product` with the fields `name` (`String`), `price` (`double`) and `expirationDate` (`LocalDate`). Write a class implementing the `Comparator` interface that compares products based on expiration date. Create a list of 5 products and sort it by expiration date.
3. Write a class `Car` with the fields `brand` (`String`), `productionYear` (`int`) and `price` (`double`). Write a class implementing the `Comparator` interface that compares cars based on production year. Create an array of 5 cars and sort it by production year.
4. Write a class `Employee` with the fields `name` (`String`), `salary` (`double`) and `hireDate` (`LocalDate`). Write a class implementing the `Comparator` interface that compares employees based on salary. Create an array of 5 employees and sort it by salary.
5. Write a class `Book` with the fields `title` (`String`), `price` (`double`) and `publicationDate` (`Date`). Write a class implementing the `Comparator` interface that compares books based on publication date. Create a list of 5 books and sort it by publication date.
6. Write a class `Product` with the fields `id` (of type `int`), `name` (of type `String`) and `price` (of type `double`). Implement the generic `Comparator` interface to compare objects by the `price` field (from the lowest to the highest price), and in case of a tie by the `id` field. Create a list of 5 objects of the `Product` class and sort it according to the described criterion.
7. Write a class `Person` with the fields `firstName` (of type `String`), `lastName` (of type `String`) and `birthDate` (of type `LocalDate`). Implement the generic `Comparator` interface to compare objects by the `lastName` field (alphabetically from A to Z), and in case of a tie by the `firstName` field. Create an array of 5 objects of the `Person` class and sort it according to the described criterion.
8. Write a class `Order` with the fields `id` (of type `int`), `customerName` (of type `String`) and `orderDate` (of type `LocalDate`). Implement the generic `Comparator` interface to compare objects by the `orderDate` field (from the earliest to the latest date), and in case of a tie by the `id` field. Create a list of 5 objects of the `Order` class and sort it according to the described criterion.

9. Write a class `Song` with the fields `title` (of type `String`), `artist` (of type `String`) and `duration` (of type `int`). Implement the generic `Comparator` interface to compare objects by the `duration` field (from the shortest to the longest song), and in case of a tie by the `title` field. Create an array of 5 objects of the `Song` class and sort it according to the described criterion.

10. Write a class `Student` with the fields `id` (of type `int`), `name` (of type `String`) and `averageGrade` (of type `double`). Implement the generic `Comparator` interface to compare objects by the `averageGrade` field (from the highest to the lowest grade average), and in case of a tie by the `name` field. Create a list of 5 objects of the `Student` class and sort it according to the described criterion.

11. Write a class `Product` with the fields `id` (of type `int`), `name` (of type `String`) and `price` (of type `double`). Implement two classes implementing the generic `Comparator` interface: `PriceComparator` to compare objects by the `price` field (from the lowest to the highest price) and `NameComparator` to compare objects by the `name` field (alphabetically from A to Z). Create a list of 5 objects of the `Product` class and sort it according to both criteria (first by price, and then, in case of a tie, by name).

12. Write a class `Person` with the fields `firstName` (of type `String`), `lastName` (of type `String`) and `birthDate` (of type `LocalDate`). Implement two classes implementing the generic `Comparator` interface: `LastNameComparator` to compare objects by the `lastName` field (alphabetically from A to Z) and `BirthDateComparator` to compare objects by the `birthDate` field (from the oldest to the youngest person). Create an array of 5 objects of the `Person` class and sort it according to both criteria (first by last name, and then, in case of a tie, by date of birth).

13. Write a class `Order` with the fields `id` (of type `int`), `customerName` (of type `String`) and `orderDate` (of type `LocalDate`). Implement two classes implementing the generic `Comparator` interface: `OrderDateComparator` to compare objects by the `orderDate` field (from the earliest to the latest date) and `CustomerNameComparator` to compare objects by the `customerName` field (alphabetically from A to Z). Create a list of 5 objects of the `Order` class and sort it according to both criteria (first by order date, and then, in case of a tie, by customer name).

14. Write a class `Song` with the fields `title` (of type `String`), `artist` (of type `String`) and `duration` (of type `int`). Implement two classes implementing the generic `Comparator` interface: `DurationComparator` to compare objects by the `duration` field (from the shortest to the longest song) and `ArtistTitleComparator` to compare objects by the `artist` field (alphabetically from A to Z) and, in case of a tie, by the `title` field. Create an array of 5 objects of the `Song` class and sort it according to both criteria (first by track length, and, in case of a tie, by artist and title).

15. Write a class `Student` with the fields `id` (of type `int`), `name` (of type `String`) and `averageGrade` (of type `double`). Implement two classes implementing the generic `Comparator` interface: `AverageGradeComparator` to compare objects by the

`averageGrade` field (from the highest to the lowest grade average) and `IdComparator` to compare objects by the `id` field (from the lowest to the highest identifier). Create a list of 5 objects of the `Student` class and sort it according to both criteria (first by grade average, and then, in case of a tie, by identifier).

22 Copying Objects

1. Write a class `Student` with three fields: `name` (String), `age` (int) and `grade` (double). Implement the `Cloneable` interface and override the `clone()` method to be able to clone objects of this class. In the `main()` method, create a `Student` object, clone it, and then change the grade (`grade`) of the original student. Display the grades of both students to see whether they are independent.

2. Write a class `Teacher` with three fields: `name` (String), `subject` (String) and `experience` (int). Implement the `Cloneable` interface and override the `clone()` method to be able to clone objects of this class. In the `main()` method, create a `Teacher` object, clone it, and then change the experience (`experience`) of the original teacher. Display the experience of both teachers to see whether they are independent.

3. Write a class `Car` with three fields: `make` (String), `model` (String) and `mileage` (double). Implement the `Cloneable` interface and override the `clone()` method to be able to clone objects of this class. In the `main()` method, create a `Car` object, clone it, and then change the mileage (`mileage`) of the original car. Display the mileage of both cars to see whether they are independent.

4. Write a class `Smartphone` with three fields: `brand` (String), `model` (String) and `productionDate` (of type `Date`). Implement the `Cloneable` interface and override the `clone()` method to be able to clone objects of this class. In the `main()` method, create a `Smartphone` object, clone it, and then change the production date of the original smartphone. Display the production date of both smartphones to see whether they are independent.

5. Write a class `Laptop` with three fields: `brand` (String), `model` (String) and `purchaseDate` (of type `Date`). Implement the `Cloneable` interface and override the `clone()` method to be able to clone objects of this class. In the `main()` method, create a `Laptop` object, clone it, and then change the purchase date (`purchaseDate`) of the original laptop. Display the purchase date of both laptops to see whether they are independent.

6. Write a class `VideoGame` with three fields: `title` (String), `genre` (String) and `releaseDate` (of type `Date`). Implement the `Cloneable` interface and override the `clone()` method to be able to clone objects of this class. In the `main()` method, create a `VideoGame` object, clone it, and then change the release date (`releaseDate`) of the original game. Display the release date of both games to see whether they are independent.

7. Write a class `CreditCard` with three fields: `cardNumber` (String), `holderName` (String) and `expiryDate` (of type `LocalDate`). Implement the `Cloneable` interface and override the `clone()` method to be able to clone objects of this class. In the `main()` method, create a `CreditCard` object, clone it, and then change the expiry date (`expiryDate`) of the original credit card. Display the expiry date of both cards to see whether they are independent.

8. Write a class `BankAccount` with three fields: `accountNumber` (String), `accountHolder` (String) and `openingDate` (of type `LocalDate`). Implement the `Cloneable` interface and override the `clone()` method to be able to clone objects of this class. In the `main()` method, create a `BankAccount` object, clone it, and then change the opening date (`openingDate`) of the original bank account. Display the opening date of both accounts to see whether they are independent.

9. Write a class `DrivingLicense` with three fields: `licenseNumber` (String), `holderName` (String) and `issueDate` (of type `LocalDate`). Implement the `Cloneable` interface and override the `clone()` method to be able to clone objects of this class. In the `main()` method, create a `DrivingLicense` object, clone it, and then change the issue date (`issueDate`) of the original driving license. Display the issue date of both driving licenses to see whether they are independent.

10. Write a class `Employee` with two fields: `name` (String) and `salaries` (an array of 12 variables of type double, representing the earnings for each month). Implement the `Cloneable` interface and override the `clone()` method to be able to clone objects of this class. In the `main()` method, create an `Employee` object, clone it, and then change the earnings at position 5 (June) of the original employee. Display the earnings of both employees to see whether they are independent.

11. Write a class `Athlete` with two fields: `name` (String) and `times` (an array of 5 variables of type double, representing the time in seconds needed to run 100 meters in different attempts). Implement the `Cloneable` interface and override the `clone()` method to be able to clone objects of this class. In the `main()` method, create an `Athlete` object, clone it, and then change the time at position 3 of the original athlete. Display the times of both athletes to see whether they are independent.

12. Write a class `Teacher` with two fields: `name` (String) and `studentsGrades` (an array of 10 variables of type double, representing the grades of each of the 10 students). Implement the `Cloneable` interface and override the `clone()` method to be able to clone objects of this class. In the `main()` method, create a `Teacher` object, clone it, and then change the grade at position 10 of the original teacher. Display the grades of both teachers to see whether they are independent.

13. Write a class `Employee` with two fields: `name` (String) and `monthlyHours` (an array list of variables of type int, representing the number of hours worked in each month). Implement the `Cloneable` interface and override the `clone()` method to be able to clone objects of this class. In the `main()` method, create an `Employee` object, clone it, and then change

the number of hours at position 5 (June) of the original employee. Display the number of hours of both employees to see whether they are independent.

14. Write a class `Athlete` with two fields: `name` (String) and `lapTimes` (an array list of variables of type int, representing the time in seconds needed to run a lap in different attempts). Implement the `Cloneable` interface and override the `clone()` method to be able to clone objects of this class. In the `main()` method, create an `Athlete` object, clone it, and then change the time at position 3 of the original athlete. Display the times of both athletes to see whether they are independent.

15. Write a class `Teacher` with two fields: `name` (String) and `studentsGrades` (an array list of variables of type int, representing the grades of each of the students). Implement the `Cloneable` interface and override the `clone()` method to be able to clone objects of this class. In the `main()` method, create a `Teacher` object, clone it, and then change the grade at position 10 of the original teacher. Display the grades of both teachers to see whether they are independent.

16. Write a class `Teacher` with the fields `name` (String), `age` (int) and `salary` (double). Then write a class `HeadTeacher` that inherits from the `Teacher` class and adds a field `bonus` (double). Implement the `Cloneable` interface and override the `clone()` method in both classes. In the `main()` method, show an example demonstrating that objects of these classes are cloned correctly.

17. Write a class `Developer` with the fields `name` (String), `age` (int) and `salary` (double). Then write a class `SeniorDeveloper` that inherits from the `Developer` class and adds a field `bonus` (double). Implement the `Cloneable` interface and override the `clone()` method in both classes. In the `main()` method, show an example demonstrating that objects of these classes are cloned correctly.

18. Write a class `Nurse` with the fields `name` (String), `age` (int) and `salary` (double). Then write a class `HeadNurse` that inherits from the `Nurse` class and adds a field `bonus` (double). Implement the `Cloneable` interface and override the `clone()` method in both classes. In the `main()` method, show an example demonstrating that objects of these classes are cloned correctly.

19. Perform the following steps:

- Create a class `Engine` with the fields `power`, `type` (engine type) and `serialNumber`. Add a parameterized constructor and implement the `Cloneable` interface.
- Create a class `Car`. The `Car` class should have the fields `make`, `model` and `engine` of type `Engine`. Add a parameterized constructor and implement the `Cloneable` interface.

23 Interfaces

1. Write an interface named `LoudAnimal` that will have one method named `makeNoise()`. Then create two classes, `Dog` and `Cat`, that will implement this interface. For each class, implement the `makeNoise()` method so that it prints the appropriate animal sound.
2. Create an interface named `MyComparator` that will contain a method `compare(int a, int b): int`. This method should return -1 if `a < b`, 0 if `a == b` and 1 if `a > b`. Then create a class `TestMyComparator` that will implement this interface. In this class, implement the `compare` method according to its description.
3. Suppose we have an interface `MusicPlayer` with the methods `turnOn()`, `turnOff()` and `nextTrack()`. Create a class `Radio` that will implement this interface. The `turnOn()` method should print the message "Radio turned on", the `turnOff()` method - "Radio turned off", and `nextTrack()` - "Radio station changed".
4. Write an interface `Converter` with three abstract methods: `convertToEuro(double amount)` returning `double`, `convertToUSD(double amount)` returning `double`, and `getConversionRate(String currency)` taking the name of a currency as a `String` and returning `double`. Create two classes, `CurrencyConverter` and `UnitConverter`, that implement this interface. In a separate class `ConverterTest`, test the implementation of the methods for objects of both classes.
5. Create an interface `Authentication` with three abstract methods: `login(String username, String password)` returning `boolean`, `logout()` returning `void`, and `resetPassword(String username, String oldPassword, String newPassword)` returning `boolean`. Create two classes, `UserAuthentication` and `AdminAuthentication`, that implement this interface. In the `AuthenticationTest` class, test the implementations of the methods for objects of both classes.
6. Design an interface `Sensor` with three abstract methods: `readValue()` returning `double`, `getStatus()` returning `String`, and `reset()` returning `void`. Create two classes, `TemperatureSensor` and `PressureSensor`, that implement this interface. In the `SensorTest` class, test how the methods work for objects of both classes.
7. Develop an interface `MediaPlayer` with three abstract methods: `play(String trackName)` returning `void`, `pause()` returning `void`, and `getCurrentTrack()` returning `String`. Create two classes, `AudioPlayer` and `VideoPlayer`, that implement this interface. In a separate class `MediaPlayerTest`, check how the methods work for objects of both classes.

8. Create an interface `Storage` with three abstract methods: `save(String data)` returning `boolean`, `delete(String identifier)` returning `boolean`, and `retrieve(String identifier)` returning `String`. Design two classes, `FileStorage` and `DatabaseStorage`, that implement this interface. In the `StorageTest` class, test the methods for objects of both classes.

9. Perform the following steps:

- Write an interface `DataProcessor` with two abstract methods: `processData(String data)` returning `String` and `isValid(String data)` returning `boolean`.
- Create a class `TextProcessor` that implements `DataProcessor`. In the `processData` method, return the data in reverse order, and in the `isValid` method, check that the data is not an empty string. Make the methods null-safe.
- Create a second class `NumberProcessor` that also implements `DataProcessor`. In the `processData` method, return the data with the additional text "Processed:" at the beginning, and in the `isValid` method, check whether the data is a number.
- In the `TestDataProcessor` class, create objects of both classes and test their methods.

Hint: To check that a string is a number, you can use a regular expression:

```
str.matches("-?\\d+(\\.\\d+)?")
```

10. Perform the following steps:

- Create an interface `VehicleManager` with two abstract methods: `startEngine()` returning `String` and `getFuelLevel()` returning `int`.
- Create a class `Car` implementing `VehicleManager`. In the `startEngine` method return "Car engine started", and in `getFuelLevel` return the value 50.
- Create a class `Motorcycle`, also implementing `VehicleManager`. In `startEngine` return "Motorcycle engine started", and in `getFuelLevel` return the value 30.
- In the `VehicleManagerTest` class, create objects of both classes and test their methods.

11. Perform the following steps:

- Create an interface `AnimalSound` with one method `makeSound()`.
- Extend this interface by creating an interface `DomesticAnimalSound` that adds a method `makeHappySound()`.
- Create a class `Dog` that implements `DomesticAnimalSound`. The `makeSound()` method should return the string "Woof", and `makeHappySound()` - "Wag tail".
- In a separate test class (`TestAnimals`), create a `Dog` object and call both methods.

12. Perform the following steps:

- Create an interface `PowerControl` with a method `turnOn()`.

- Extend this interface by creating `AdvancedPowerControl`, which adds a method `setPowerSavingMode()`.
- Create a class `SmartLamp` implementing `AdvancedPowerControl`. The `turnOn()` method should activate the lamp, and `setPowerSavingMode()` should switch it to power-saving mode (the methods should display appropriate messages).
- In a separate test class (`TestDevices`), create a `SmartLamp` object and call both methods.

13. Perform the following steps:

- Create an interface `Drawable` with a method `draw()`.
- Extend this interface by creating `ColorDrawable`, which adds a method `setColor(String color)`.
- Create a class `Circle` implementing `ColorDrawable`. The `draw()` method should draw a circle, and `setColor()` should change the color of the circle (the methods should display appropriate messages).
- In a separate test class (`TestDrawing`), create a `Circle` object, set the color and draw the circle using both methods.

14. Create an interface `Drawable` with:

- An abstract method `draw()`.
- A default method `display()` that displays the message "Displaying Drawable".
- A static method `getType()` that returns the `String` "Drawable Type".

Create the classes `Circle` and `Rectangle` that implement `Drawable`. `draw()` in `Circle` should display "Drawing Circle", and in `Rectangle` - "Drawing Rectangle". Create a test class `DrawableTester`. Create `Circle` and `Rectangle` objects, call `draw()` and `display()` on them, and call `Drawable.getType()` statically.

15. Create an interface `SoundPlayer` with:

- An abstract method `playSound()`.
- A default method `stopSound()` that displays the message "Sound Stopped".
- A static method `getDeviceType()` that returns the `String` "Sound Device".

Create the classes `MusicPlayer` and `Radio` that implement `SoundPlayer`. `playSound()` in `MusicPlayer` should display "Playing Music", and in `Radio` - "Playing Radio". Create a test class `SoundTester`. Create `MusicPlayer` and `Radio` objects, call `playSound()` and `stopSound()` on them, and call `SoundPlayer.getDeviceType()` statically.

16. Create an interface `MemoryManager` with:

- An abstract method `allocateMemory(int size)`.
- A default method `freeMemory()` that displays the message "Memory Freed".
- A static method `getMemoryType()` that returns the `String` "Memory Type".

Create the classes `RAMManager` and `DiskManager` that implement `MemoryManager`. `allocateMemory(int size)` in `RAMManager` should display "Allocating RAM Memory", and in `DiskManager` - "Allocating Disk Space". Create a test class `MemoryTester`. Create `RAMManager` and `DiskManager` objects, call `allocateMemory(int size)` and `freeMemory()` on them, and call `MemoryManager.getMemoryType()` statically.

17. Create an interface `VATCalculator` with:

- A field `double vatRate` set to the standard VAT rate (e.g. 23%).
- A static method `calculateWithVAT(double price)` that calculates and returns the price of a product including VAT.

Create a test class `VATTest` that calls the `calculateWithVAT(double price)` method with a sample product price and displays the result.

24 Exceptions

1. Write a program that defines a method `checkAge(int age)`. This method should throw an `IllegalArgumentException` with an appropriate message if the given age is less than 18. In the main method of the program (`main`), call `checkAge` with various values and handle the exception by displaying an appropriate message to the user.
2. Write a program that asks the user to enter two numbers and then divides the first number by the second. The program should handle two kinds of exceptions: `ArithmeticException` in the case of division by zero and `InputMismatchException` when the user enters something other than numbers. In both cases, display an appropriate error message and ask the user to enter the data again. Use the `int` type.
3. Design and implement an exception class `InvalidDataFormatException` that extends the `Exception` class. Then write a method `checkDataFormat(String data)` that throws an `InvalidDataFormatException` if the given string does not match a specific pattern (e.g. it is not an e-mail address). In the `main` method, test how this method works by handling the exception and informing the user about the error.

25 Generic Programming

1. Create a simple generic class `Box` that can store an object of any type. The class should contain a method `set` to set the object and a method `get` to retrieve it.
2. Write a generic method `isEqual` that takes any two objects of the same type and returns `true` if they are equal, otherwise `false`.
3. Create a generic class `Counter<T>` that will count the number of added elements of a given type. The class should have a method `add(T element)` that adds an element to an internal structure, and a method `getCount()` that returns the number of added elements.
4. Create a generic class `Triple<T, U, V>` that can store three objects of different types. Implement the methods `getFirst()`, `getSecond()` and `getThird()` to retrieve the first, second and third element, respectively.
5. Write a generic method `max` that takes an array of elements of a comparable type (implementing the `Comparable<T>` interface) and returns the element with the highest value. Handle the case of an empty array.
6. Define a generic interface `Stack<T>` with the methods `push(T item)`, `T pop()`, `T peek()` and `boolean isEmpty()`. Create a class implementing this interface that will represent a stack storing elements of any type.
7. Create a generic class `Storage<T>` that stores a single object of any type. The class should have the methods `store(T item)`, which saves the object, and `T retrieve()`, which returns the stored object.
8. Write a generic method `printArray` that takes an array of elements of any type and prints all the elements of the array to standard output. The method should be able to handle arrays of any object type.
9. Create a generic class `Pair` that stores two values of any type. The class should have two methods: `getFirst()`, which returns the first element of the pair, and `getSecond()`, which returns the second element of the pair.
10. Write a generic method `swap` that takes an array of any type and two integers representing indices. The method should swap the elements of the array at the given indices.
11. Create a generic class `GenericQueue<T>` that implements a simple queue. The class should have the methods `enqueue(T element)`, which adds an element to the queue, and `dequeue()`, which removes and returns the element from the front of the queue.

12. Write a static generic method `swap` that takes an array of any type and two indices, and then swaps the elements in this array at the specified indices. The method should work for an array of any type. Sample method call: `swap(myArray, 0, 2);`, where `myArray` is an array of type `Integer[]` or any other type. Safeguard the method so that it cannot be called with indices outside the range of the array.

13. Create a static generic method `reverseArray` that takes an array of elements of type `T` and reverses the order of its elements. The method should modify the passed array without returning a new one. Check how the method works on arrays of different types, such as `Character`, `Boolean` and your own object types. Safeguard the method so that it cannot be called with a `null` array.

14. Write a static generic method `minValue` that takes an array of elements of a generic type `T`, where `T` extends `Comparable<T>`. The method should return the smallest element of the array. Test this method on arrays containing different types of comparable objects, such as `Integer`, `Double` or `String`. Safeguard the method so that it cannot be called with an array with zero elements. Add a class `Person` with the fields `name` and `age` and test the `minValue` method on an array of `Person` objects.

15. Write a static generic method `maxValue` that takes an array of elements of a generic type `T`, where `T` extends `Comparable<T>`. The method should return the largest element of the array. Make sure that the method does not accept an empty array (with zero elements). Test the method on arrays containing different types of comparable objects, such as `Integer`, `Float` or `String`. Create a class `Vehicle` with the fields `model` and `speed`, implementing the generic `Comparable`, and test the `maxValue` method on an array of `Vehicle` objects.

16. Create a static generic method `sortAndReturnFirst` that takes an array of elements of a generic type `T`, where `T` extends `Comparable<T>`. The method should sort the array and return its first element. Safeguard the method against being called with an empty array (with zero elements). Test this method on various data types, including a newly created class `Book` with the fields `title` and `author`. The `Book` class should implement the generic `Comparable` interface based on the `title` field according to the natural ordering.

17. Write a static generic method `printArray` that takes as an argument an array of elements of a generic type `T` and prints all the elements of the array. Test this method on arrays of type `Integer` and `Double`. Then try to test the method on an array of a primitive type, e.g. `int` or `double`, and note the resulting compilation errors. Explain why Java does not allow primitive types to be used in the context of generic types.

18. Create a static generic method `sumElements` that takes a collection of elements of a generic type `T` and returns their sum. Test this method on collections of type `Integer` and `Float`. Then try to use this method with a collection of primitive types, e.g. `int` or `float`, and see what compilation errors appear. Use this to explain the difference between primitive types and object types in Java.

19. Design a static generic method `compareElements` that takes two arguments of a generic type `T` and returns a `boolean` value indicating whether they are equal. Test the method on pairs of `Integer`, `String` and `Character` objects. Then try to test the same method on primitive types, such as `int` or `char`, and describe the problems encountered when trying to use primitive types with generics.

20. Write a static generic method `printObjectInfo` that takes an argument of a generic type `T` with an upper bound of `Object`. The method should print the object's class, its hashcode and the result of the `toString()` method. Test this method on various types of objects, such as `String`, `Integer`, and your own classes, e.g. `Car`, `Animal`, where each class has its own unique fields and methods.

21. Create two classes: `Animal` and `Dog`, where `Dog` inherits from `Animal`. Then write a static generic method `findMax` that takes two arguments: `element1` and `element2` of a type that `extends Animal`. The method should return the larger element according to a comparison criterion you define yourself. In the implementation, compare the elements based on an attribute of your choice, for example age.

22. Define the classes `Car` and `ElectricCar`, where `ElectricCar` inherits from `Car`. Write a static generic method `compareObjects` that takes two arguments: `object1` and `object2` of a type that `extends Car`. The method should return `true` if the objects are of the same type, otherwise `false`. Use the `getClass()` method to compare the classes of the objects.

23. Create a generic class `Pair<T>` that stores two objects of the same type. Create two classes: `Animal` and `Dog`, where `Dog` inherits from `Animal`. Then write a static generic method `findMax` that takes a `Pair<? extends Animal>`. The method should return the larger element according to a comparison criterion you define yourself.

24. Create a generic class `Triple<T>` that stores three objects of the same type. Then create two classes: `Bird` and `Eagle`, where `Eagle` inherits from `Bird`. Then write a static generic method `findMin` that takes a `Triple<? extends Bird>`. This method should return the smallest element based on a comparison criterion you define.

25. Create a generic class `Triple<T>` that stores three objects of the same type. Then create two classes: `Bird` and `Eagle`, where `Eagle` inherits from `Bird`. Then write a static generic method `findMin` that takes a `Triple<? extends Bird>`. This method should return the smallest element based on a comparison criterion you define.

26. Create a generic class `ElementPair<T>` that stores two objects of the same type. Create two classes: `Shape` and `Circle`, where `Circle` inherits from `Shape`. Then write a static generic method `findLargest` that takes an `ElementPair<? extends Shape>`. The method should return the larger element according to a comparison criterion you define yourself.

27. Create a generic class `DoubleElement<T>` that stores two objects of the same type. Create two classes: `Person` and `Student`, where `Student` inherits from `Person`. Then write a static generic method `findYoungest` that takes a `DoubleElement<? extends Person>`. The method should return the smaller element according to your own comparison criterion, for example age.

28. Create a generic class `Pair<T>` that stores two objects of the same type. Create two classes: `Animal` and `Dog`, where `Dog` inherits from `Animal`. The `Dog` class should have a private field `age` that the `Animal` class does not have. Then write a static generic method `findMinMaxAge` that takes as arguments an array of objects of type `Dog` and a `Pair<?super Dog> result`. The method should store (as objects of type `Dog`) the youngest and the oldest dog (in terms of age) from the array in the second argument of the method. Do this directly, without using the `Comparable` or `Comparator` interfaces.

29. Create a generic class `Pair<T>` that stores two objects of the same type. Then create two classes: `Plant` and `Tree`, where `Tree` inherits from `Plant`. The `Tree` class should have a private field `height` that the `Plant` class does not have. Then write a static generic method `findMinMaxHeight` that takes as arguments an array of objects of type `Tree` and a `Pair<? super Tree> result`. The method should store (as objects of type Tree) the shortest and the tallest tree (in terms of height) from the array in the second argument of the method. Also use the generic `Comparable` interface.

26 Collections

26.1 The `Collection` Interface

A1. Write a static method `printUnique(Collection<T> items)` that takes the generic Collection interface as an argument and displays each unique element of this collection on the screen exactly once.

A2. Create a static method `countOccurrences(Collection<T> items, T element)` that returns the number of occurrences of the given element in the given collection. The function should work for any type of object stored in the collection.

A3. Implement a method `removeEveryOther(Collection<T> items)` that removes every other element from the passed collection. The method should modify the original collection without creating a copy of it.

26.2 The `Iterable` Interface

B1. Write a method `reversePrint(Iterable items)` that takes the generic Iterable interface as an argument and displays the elements of this sequence on the screen in the reverse order to that in which they were passed.

B2. Create a function `findMax(Iterable numbers)` that searches an Iterable collection containing numbers and returns the largest number. We assume that the elements of the collection are `Comparable` objects.

B3. Implement a method `countElements(Iterable items, Object element)` that counts how many times a given element appears in a collection implementing the Iterable interface. The method should compare elements using the `equals` method.

26.3 The `ArrayList` Array List

C1. Create a method `mergeLists` that takes two generic `ArrayList<T>` and returns a new `ArrayList<T>` that is the concatenation of the elements of both lists. Make sure that the order of the elements from the original lists is preserved in the resulting list.

C2. Write a function `removeDuplicates` that takes an `ArrayList<T>` and returns a new list from which all duplicates have been removed, leaving only unique elements. The order of the retained elements should correspond to their first occurrence in the original list.

C3. Implement a method `countOccurrences` that takes an `ArrayList<T>` and an element of type `T`, and then returns the number of occurrences of this element in the given list.

26.4 The `LinkedList` Linked List

D1. Write a method `isPalindrome` that takes a generic linked list (`LinkedList<T> list`) and returns `true` if the list is a palindrome, and `false` otherwise. A list is a palindrome if it reads the same forwards and backwards. The method should be as efficient as possible.

D2. Write a method `findCommonElements` that takes two generic linked lists (`LinkedList<T> list1` and `LinkedList<T> list2`) and returns a new list containing the elements that occur in both `list1` and `list2`. The elements in the returned list should be unique and do not need to be sorted. The method should not modify the input lists.

26.5 The Hash-Table-Based Set `HashSet`

E1. Write a method `findUniqueElements` that takes a generic list (`List<T> list`) and returns a `HashSet<T>` that contains only the unique elements of this list. The method should effectively eliminate duplicates.

E2. Write a method `hasCommonElements` that takes two generic sets (`HashSet<T> set1` and `HashSet<T> set2`) and returns `true` if both sets have at least one element in common, and `false` otherwise.

E3. Write a method `unionSets` that takes two generic collections of type `HashSet<T>` (`HashSet<T> set1` and `HashSet<T> set2`) and returns a new set (`HashSet<T>`) that is the union of both input sets. The resulting set should contain all the elements that occur in `set1`, `set2`, or in both of these sets.

26.6 `TreeSet`

F1. Write a method `findElementsInRange` that takes a `TreeSet<T>` and two bound values `T lowerBound` and `T upperBound`. The method should return a `TreeSet<T>` that contains all the elements of the original set that fall within the interval between `lowerBound` and `upperBound` (inclusive of both bounds).

F2. Write a method `removeMinMax` that takes a `TreeSet<T>` and modifies it by removing the smallest and the largest element. The method should return a pair (e.g. a `Pair` class or a two-element array/list) containing the removed values. If the set has fewer than two elements, the method should return appropriate null values or indicate an error.

F3. Write a method `findClosestElement` that takes a `TreeSet<T>` and a value `T target`. The method should return the element of the set that is closest to the value `target`. If two elements are equally close, the method may return either of them. We assume that the type `T` allows values to be compared (e.g. by implementing the `Comparable<T>` interface).

26.7 `Queue`

G1. Write a method `reverseQueue` that takes a generic queue (`Queue<T> queue`) and reverses the order of its elements. The method should perform the reversal in place, without using additional collections.

G2. Write a method `simulateSupermarketQueue` that takes a generic queue (`Queue<Customer> customers`) and a positive integer `n` representing the number of available checkouts. The `Customer` class contains the customer's service time. The method should return the total time needed to serve all customers in the queue, assuming that each checkout serves one customer at a time and takes the next customer from the front of the queue as soon as it becomes free.

26.8 `Deque`

H1. Write a method `isSymmetric` that takes a generic Deque (`Deque<T> deque`) and returns `true` if the elements in the Deque are arranged symmetrically (i.e. the order of the elements is the same whether we read them from front to back or from back to front). Otherwise, the method returns `false`.

H2. Write a method `removeEverySecondElement` that takes a generic Deque (`Deque<T> deque`) and removes every second element from it, starting with the second element in the order in which the elements were added. The method should modify the original Deque and return nothing.

H3. Write a method `swapEnds` that takes a generic Deque (`Deque<T> deque`) and swaps the first element with the last one. If the Deque contains fewer than two elements, the method should do nothing. The method should modify the original Deque and return nothing.

26.9 The `PriorityQueue` Priority Queue

I1. Write a method `mergePriorityQueues` that takes two generic PriorityQueues (`PriorityQueue<T> queue1` and `PriorityQueue<T> queue2`). The method should return a new PriorityQueue containing all the elements of both input queues. We assume that the elements in both queues can be compared with each other.

26.10 `Map`

J1. Write a method `reverseMap` that takes a generic map (`Map<K, V> map`) and returns a new map (`Map<V, K>`), where each key becomes a value and each value becomes a key. If the original map contains repeated values, keep only the last key-value pair when reversing the map.

J2. Write a method `findKeysWithMaxValue` that takes a generic map (`Map<K, V> map`), where the values are comparable (implement `Comparable<V>`). The method should return a list of all keys that are associated with the largest value in the map.

J3. Write a method `groupByValue` that takes a generic map (`Map<K, V> map`) and returns a new map (`Map<V, List<K>>`), where each value from the original map becomes a key in the new map, and its value is the list of keys that were associated with it in the original map.

26.11 `HashMap`

K1. Write a method `countValueOccurrences` that takes a generic HashMap (`HashMap<K, V> map`) and returns a new HashMap (`HashMap<V, Integer>`), where each key is a value from the original map, and the value is the number of occurrences of that value in the original map.

K2. Write a method `swapKeysAndValues` that takes a generic HashMap (`HashMap<K, V> map`) and returns a new HashMap (`HashMap<V, K>`), where each key from the original map becomes a value and each value becomes a key. We assume that all values in the original map are unique.

K3. Write a method `aggregateValuesByKey` that takes a generic HashMap (`HashMap<K, Integer> map`) as well as a key (`K key`) and a value (`Integer value`). The method should add the value to the current value associated with the key. If the key does not exist in the map, it should be added with the given value. The method returns nothing and modifies the original map.

K4. Write a method `compareMaps` that takes two generic HashMaps (`HashMap<K, V> map1` and `HashMap<K, V> map2`) and returns `true` if both maps have exactly the same key-value pairs, and `false` otherwise. The method should compare both keys and values.

26.12 `TreeMap`

L1. Write a method `subMapInRange` that takes a generic TreeMap (`TreeMap<K, V> map`) as well as two key values `K startKey` and `K endKey`. The method should return a new TreeMap that contains all key-value pairs from the original map whose keys fall within the range from `startKey` to `endKey`, inclusive.

L2. Write a method `reverseOrderMap` that takes a generic TreeMap (`TreeMap<K, V> map`). The method should return a new TreeMap that contains all key-value pairs from the original map, but with the order of the keys reversed.

Hint: use `Collections.reverseOrder()`.

L3. Write a method `calculateAverageValue` that takes a generic TreeMap (`TreeMap<K, Double> map`). The method should calculate and return the average of all values (of type Double) stored in the map. We assume that the map is not empty.

26.13 `Vector`

M1. Create a function `concatenateVectors` that takes two `Vector<T>` objects and returns a new `Vector<T>` containing all the elements of the first vector, followed by all the elements of the second vector.

M2. Write a method `reverseVector` that takes a `Vector<T>` object and returns a new `Vector<T>` in which the order of the elements is reversed relative to the original vector.

M3. Implement a function `filterVector` that takes a `Vector<T>` and the functional interface `Predicate<T>`. The function should return a new `Vector<T>` containing only those elements of the original vector that satisfy the condition defined by `Predicate<T>`.

27 Delegation

1. Perform the following steps:

a. Create an interface `Printer` with a method `print(String text)`.
b. Create a class `StandardPrinter` that implements `Printer` and prints the text to the console.
c. Create a class `Office` that has a private field of type `Printer`. In the `Office` constructor, take a `Printer` as an argument and assign it to the field. Add a method `printDocument(String text)` that will delegate the printing task to the `Printer` object.

2. Perform the following steps:

a. Create an interface `Engine` with the methods `start()` and `stop()`.
b. Create a class `PetrolEngine` that implements `Engine` and simulates the operation of a petrol engine.
c. Create a class `Car` that has a private field of type `Engine`. In the constructor, take an `Engine` as an argument. Add the methods `start()` and `stop()`, which will delegate the tasks of starting and stopping the engine, respectively, to the `Engine` object.

3. Perform the following steps:

a. Create an interface `Notification` with a method `send(String message)`.
b. Create a class `Email` that implements `Notification` and simulates sending an e-mail message.
c. Create a class `User` that has a private field of type `Notification`. In the constructor, take a `Notification` as an argument. Add a method `notifyAboutUpgrade(String information)` that will delegate the task of sending the notification to the `Notification` object.

Part III

Miscellaneous Exercises

Part IV

Design Patterns

References and Other Exercise Collections